



Генерисање описа слике применом LSTM и ResNet модела

Мастер рад

Анђа Денић

Број индекса: 110 РН

Департман за рачунарске науке
Природно-математички факултет
Универзитет у Нишу

Ментор: др Александар Трокицић

4. новембар 2025.
Ниш, Србија

Апстракт

Аутоматско генерисање описа слике је фундаментални проблем у вештачкој интелигенцији који повезује рачунарски вид (енгл. *Computer Vision*, *CV*) и обраду природног језика (енгл. *Natural Language Processing*, *NLP*). У овом раду је представљен генеративни модел заснован на енкодер-декодер архитури, при чему је енкодер *ResNet*, а декодер *LSTM* модел. Модел је трениран да за дату слику на улазу максимизира веродостојност анотације на излазу. Експериментално је показано да модел трениран на *MC COCO* скупу података генерише описе високих перформанси. Модел достиже следеће вредности евалуационих метрика на скупу података за тестирање: 68.36 BLEU-1, 50.35 BLEU-2, 35.70 BLEU-3, 25.23 BLEU-4, 49.51 ROUGE-L, 82.62 CIDEr.

Кључне речи: **Енкодер-декодер, MC COCO, ResNet, LSTM, Computer Vision, NLP**

Захвалнице

Највећу захвалност дугујем мојој сестри, Невени Денић, момку Стефану Стојановићу и пријатељу Николи Васићу на помоћи при писању овог мастер рада и имплентацији. Они су увек били доступни за вишечасовне консултације у пољима својих експертиза, али и да ми пруже емотивну подршку током мастер студија.

Желела бих да се захвалим свом ментору, професору доктору Александру Трокицићу, на менторству и слободи коју ми је допустио током писања мастер рада. Такође сам захвална професору доктору Бранимиру Тодоровићу и асистентима Вукашину Станојевићу и Јелени Пејић од којих сам много научила и који су ме инспириали да наставим да се бавим машинским учењем.

На крају, моја захвалност иде мојим родитељима, Сањи Денић и Зорану Денићу који су истицали важност образовања од када знам за себе.

Садржај

Апстракт	1
Захвалнице	2
Нотација	5
1 Увод	6
1.1 Машинско учење и вештачка интелигенција	6
1.2 Мотивација	8
1.3 Опис проблема	8
1.4 Предложено решење и доприноси	9
1.5 Структура мастер рада	10
2 Позадина проблема, скупа података и модела	11
2.1 МС СОСО скуп података	11
2.2 Енкодер-декодер	14
2.2.1 Елементи енкодера-декодера	16
2.2.2 Примена Енкодера-Декодера	17
2.2.3 Градивни блокови енкодера и декодера	18
2.2.4 Тренирање енкодер-декодер модела	19
2.3 ResNet	22
2.3.1 Историјска позадина	22
2.3.2 Конволуциони блок	23
2.3.3 <i>Max Pooling</i> слој	26
2.3.4 Резидуални блок	26
2.3.5 <i>Batch Normalization</i> слој	28
2.3.6 <i>Bottleneck</i> блок у <i>ResNet-152</i>	28
2.3.7 Архитектура <i>ResNet</i> модела	28
2.4 LSTM	30
2.4.1 Моделирање секвенци	30
2.4.2 Vanilla Recurrent Neural Network	32
2.4.3 The Long Short-Term Memory	35
2.5 BLEU метрика	46
3 Методологија	50
3.1 Скуп података	50
3.1.1 Препроцесуирање слика и описа	50

3.2	Архитрктуре модела	51
3.2.1	Енкодер-декодер	51
3.2.2	Енкодер	52
3.2.3	Декодер	52
3.3	Тренирање	53
3.3.1	Параметри модела за тренирање	53
3.3.2	Пробабилистичка интерпретација модела	54
3.3.3	Метод максималне веродостојности	54
3.3.4	Градијентни спуст	56
3.4	Генерисање описа	56
3.5	Евалуација	56
4	Имплементација	57
4.1	Подаци	57
4.2	Препроцесуирање	58
4.3	Дефинисање модела	58
4.4	Тренирање модела	58
4.5	Евалуација модела	59
5	Експерименти	60
6	Закључак и будући рад	63
	Биографија кандидата	64

Нотација

Табела 1: Нотација

Промењива	Домен	Објашњење
N	$\mathbb{N}$	Обим скупа за тренирање.
(x_n, y_n)		Препроцесуирани узорак из скупа за тренирање. Пар слика-опис.
x_n	$\mathbb{R}^{3 \times 224 \times 224}$	n -та препроцесуирана слика из скупу података за тренирање
T	$\mathbb{N}$	Дужина најдужег препроцесуираног описа у скупу података за тренирање
y_n	$\mathbb{R}^T$	n -ти препроцесуирани опис у скупу података за тренирање
E	$\mathbb{N}$	Димензија ембедованог простора речи и слика
z_n	$\mathbb{R}^E$	Вектор карактеристика који је излаз енкодера $z_n = \text{Encoder}(x_n)$
V	$\mathbb{N}$	Величина вокабулара
Y_n	$T \times V$	Опис за улаз x_n . Случајна променљива.
$Y_{n,t}$	V	Реч на месту t у опису слике x_n . Случајна променљива.
$y_{n,t}$	$\{0, 1\}^V$	<i>One-hot</i> енкодована реч на месту t у анотацији y_n .
$\hat{p}_{n,t,v}$	$[0, 1]$	Вероватноћа да се на месту t у опису слике x_n нађе реч v . Предикција декодера.
θ		Параметри енкодера и декодера који сетренирају.
θ^*		Оптимални параметри који се добијају по завршетку тренирања.

Глава 1

Увод

1.1 Машинско учење и вештачка интелигенција

Вештачка интелигенција, краће ВИ (енгл. *Artificial Intelligence, AI*), је актуелна област, са многим практичним применама и активним темама за истраживање. Циљ ВИ је креирање интелигентног софтвера који аутоматизује рутинске послове, разуме говор или слике и поставља дијагнозе у медицини.

Вештачка интелигенција је најпре решавала проблеме који су интелектуално тешки за људе, али релативно једноставни за рачунаре - проблеме који се могу описати листом формалних, математичких правила. Рачунари су у стању да победе и најбољег људског играча шаха. На пример, IBM-ов систем за игру шаха "Deep Blue" је победио светског првака Гарија Каспарова 1997. године. Шах се може у потпуности описати кратком листом формалних правила, и зато рачунари постижу високе перформансе.

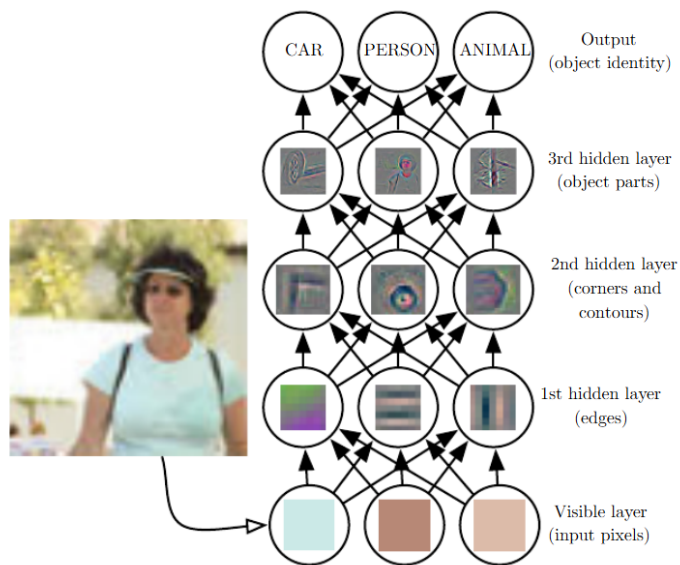
Прави изазов вештачке интелигенције је решавање свакодневних задатака које људи лако обављају, али тешко формално описују, попут препознавања изговорених речи или лица на сликама (Goodfellow, Yoshua Bengio и Courville 2016.). Описивање слике једном реченицом (енгл. *image captioning*) управо је такав проблем и овај рад покушава да га реши.

Решење је омогућити рачунарима (машинама) да уче из искуства, користећи велики скуп података. Зато се ова област назива „машинско учење“ (енгл. *Machine Learning, ML*). За решавање проблема *image captioning*-а, машина користи скуп података који садржи више од 100.000 слика и њихових описа које су ручно написали људи. Циљ је да машина научи функцију слика $\rightarrow$ опис, тако да буде у стању да произведе опис слике коју раније није видела. На тај начин, људи не морају формално да наведу сво знање које рачунар треба да поседује.

Неки аутори (Goodfellow, Yoshua Bengio и Courville 2016.) сматрају да је машинско учење једини изводљив приступ изградњи AI система који могу да функционису у стварном свету. Дубоко учење (енгл. Deep Learning) је подобласт машинског учења која даје добре резултате и флексибилност. У дубоком учењу 'свет' је представљен као угњеждена хијерархија појмова, при чему је сваки појам дефинисан у односу на једноставније појмове, а апстрактније репрезентације се рачунају на основу мање апстрактних, као што је приказано на слици 1.1.

У овом мастер раду примењен је приступ дубоког учења на задатак аутомат-

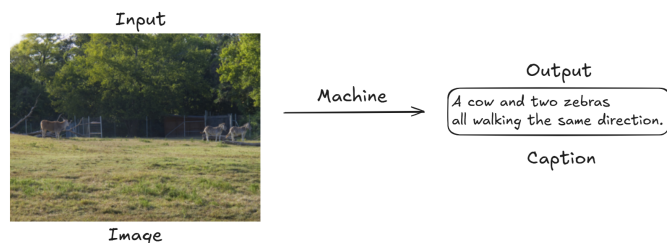
ског генерисања описа слике, боље објашњеног у наредној глави [1.2](#).



Слика 1.1: Илустрација модела дубоког учења. Рачунару је тешко да разуме значење сирових података са сензора. Пример је ова слика представљена као скуп вредности пиксела. Функција која слика скуп пиксела у идентитет објекта је веома компликована. Готово је немогуће ову функцију задати експлицитно. Дубоко учење решава овај проблем тако што разлаже жељену сложену функцију на неколико угњежених једноставнијих функција, од којих свака припада различитом слоју модела. Најједноставније функције у овој хијерархији се могу експлицитно задати. У улазном слоју се налази слика задата као низ пиксела. Затим, низ скривених слојева `екстрактује` све апстрактније карактеристике (енгл. features) из слике. Ови слојеви се називају `скривеним` јер њихове вредности нису дате у скупу података, уместо тога, модел мора да одреди које карактеристике су корисне за објашњење односа у посматраним подацима. На основу улазних пиксела, први слој може лако да идентификује ивице упоређивањем светлине суседних пиксела. На основу описа ивица из првог скривеног слоја, други скривени слој може лако препознати углове и продужене контуре, као скупови ивица. На основу описа слике у другом скривеном слоју у терминима углова и контура, трећи скривени слој може детектовати читаве делове специфичних објеката проналаском специфичних скупова контура и углова који одговарају том објекту. Рецимо, уколико је објекат лопта, трећи слој може да научи да идентификује круг. На крају, овај опис слике у терминима делова објеката које садржи може се користити за препознавање објеката присутних на слици. Извор: Goodfellow, Yoshua Bengio и Courville [2016](#).

1.2 Мотивација

Циљ овог пројекта је креирање програма који може аутоматски да генерише описе слика које му се прослеђују. За дату слику коју програм види као улаз, требало би да генерише реченицу која описује ту слику као излаз, као што је приказано на 1.2.



Слика 1.2: Машина која аутоматски генерише опис слике.

Да бисмо направили такав програм, користићемо парадигму машинског учења, што значи да функција улаз $\rightarrow$ излаз (слика $\rightarrow$ опис) неће бити експлицитно задата; уместо тога, функција ће се учити током тренирања коришћењем скупа података за тренирање.

Аутоматско генерисање описа који представљају правилно написане реченице на енглеском је веома изазован проблем. Ипак, решен проблем би има неколико примена:

- Помоћ особама са општећеним видом да боље разумеју садржај слика на вебу
- Омогућавање претраживачима слика и дигиталним библиотекама да се ефикасно организују
- Класификација визуелних података

Генерисање описа слике је тежи задатак од класификације слике или препознавања објеката на слици, који су били у фокусу заједнице рачунарског вида (Russakovsky и др. 2015.). Опис мора да обухвати не само објекте на слици, већ и однос између објеката, атрибуте објеката, активности... Штавише, опис мора да буде написан исправно коришћењем правила енглеског језика, што значи да је поред визуелног разумевања потребан и језички модел.

1.3 Опис проблема

У овом мастер раду је описана теоријска позадина модела дубоког учења који је способан да генерише текстуални опис задате слике, као и имплементација и евалуација тог модела у програмском језику Python. Специјално, модел користи ResNet модел (He и др. 2016.) као енкодер и LSTM (Hochreiter и Schmidhuber 1997.) модел као декодер. Модел је трениран и евалуиран на MC COCO (Lin и др. 2014.) скупу података.

1.4 Предложено решење и доприноси

Циљ овог рада је анализа предложеног решења датог у раду „Show and Tell: A Neural Image Caption Generator“ (Vinyals и др. 2015.).

Коришћен је натрениран (енгл. *pre-trained*) *ResNet-152* модел као енкодер, познат по својим могућностима екстракције карактеристика (енгл. *feature extraction*). Овај енкодер обрађује улазне слике, трансформишући их у векторску презентацију фиксне дужине која се назива вектор карактеристика (енгл. *feature vector*). Након тога, рекурентна неуронска мрежа (енгл. *RNN*) *LSTM* (*Long-Short Term Memory*) служи као декодер. *LSTM* узима излаз из *ResNet* енкодера као почетно стање и секвенцијално генерише опис слике, реч по реч. Читав систем је трениран од почетка до краја (енгл. *end-to-end*) на обимном скупу података *Microsoft Common Objects in Context (MS COCO)*, који је специфично дизајниран за проблеме детекције објеката на сликама, сегментације слика и генерисања описа слика. Састоји се од разноврсне колекције слика упарених са вишеструким људским анотацијама.

Кључни доприноси овог мастер рада су:

1. **Детаљно описана теоријска позадина:** Овај рад објашњава све поменуте концепте коришћене за генерисање описа слика, укључујући архитектуру енкодера-декодера, скуп података *MS COCO*, *ResNet*, *LSTM*, рекурентне неуронске мреже (*RNN*) и *BLEU* метрику за евалуацију, у глави 2
2. **Свеобухватни математички модел:** Представљена је детаљна математичка формулација проблема за генерисање описа, модела машинског учења који га решава, оптимизационог проблема, функције цене, као и коришћених алгоритама оптимизације у глави 3
3. ***Open-Source Python* имплементација:** Комплетан код пројекта је јавно доступан на *GitHub*-у <https://github.com/andjadenic/ImageCaptioning>. Овај репозиторијум садржи детаљна упутства о томе како имплементирати модел користећи *PyTorch* библиотеку за дубоко учење, олакшавајући репродуктивност и подстичући даља истраживања и развој у овој области. Детаљи око имплементације дати су у глави 4.

1.5 Структура мастер рада

Мастер рад је организован у 8 глава:

1. Глава 1 уводи појмове који се користе у мастер раду. Дефинисани су **основни појмови**: машинско учење, вештачка интелигенција и дубоко учење, као и начин на који овај рад користи поменуте парадигме у поглавље 1.1. Затим, дата је **мотивација** за рад у поглављу 1.2, као и **опис проблема** у поглављу 1.3 и **предложеног решења** у поглављу 1.4.
2. У глави 2 дат детаљан **математички преглед и историјска позадина проблема, скупа података и модела**. У поглављу 2.1 је описан **МС СОСО скуп података** који је коришћен за тренирање и евалуацију модела. Одељак 2.2 је посвећен **енкодер-декодер моделу**. Дефинисани су његови основни елементи, градивни блокови, појашњено је како треба да изгледа скуп података за тренирање енкодер-декодер модела, као и начин на који се овај модел тренира. Дате су неке примене модела данас, као и историјски преглед настајања. Одељак 2.3 је посвећен **ResNet** моделу, одељак 2.4 **LSTM** моделу, а одељак 2.5 **BLEU** метрици за евалуирање.
3. У глави 3 је изложена методологију рада. У поглављу 3.1 је описан скуп података за тренирање и алгоритми за препроцесуирање описа и слика, у поглављу 3.2 је описана архитектура модела и *forward pass*, у поглављу је описан оптимизациони проблем и алгоритам за његово решавање (тренирање модела) 3.3, у поглављу 3.4 је приказано како се истренирани модели користе за генерисање описа слике и у поглављу 3.5 је описан процес евалуације натренираног модела.
4. Детаљи везани за имплементацију су дати у глави 4. Свака фаза у пројекту је описана посебно: **Подаци** у секцији 4.1, **Препроцесуирање** у секцији 4.2, **Имплементација модела** у секцији 4.3, **Тренирање модела** у секцији 4.4 и **Евалуација модела** у секцији 4.5.
5. У глави 5 су приказани резултати модела.

Глава 2

Позадина проблема, скупа података и модела

У овој глави дат детаљан математички преглед и историјска позадина проблема, скупа података и модела. У поглављу 2.1 је описан МС СОСО скуп података који је коришћен за тренирање и евалуацију модела. Подељак 2.2 је посвећен енкодер-декодер моделу. Дефинисани су његови основни елементи, градивни блокови, појашњено је како треба да изгледа скуп података за тренирање енкодер-декодер модела, као и начин на који се овај модел тренира. Дате су неке примене модела данас, као и историјски преглед настајања. Дате су неке примене модела данас, као и историјски преглед настајања. Одељак 2.3 је посвећен ResNet моделу, одељак 2.4 LSTM моделу, а одељак 2.5 BLEU метрици за евалуирање.

2.1 МС СОСО скуп података

За развој модела машинског учења неопходни су скупови података великог обима. Разноврсност и квалитет података играју кључну улогу. Моделе са добрим перформансама су могући једино ако се тренирају на подацима високог квалитета. Један од најлакших начина да се обезбеде висококвалитетни подаци је коришћење постојећих "benchmark" скупова података као што је МС СОСО (Lin и др. 2014.).

The COCO (Common Objects in Context) је скуп података који се користи за проблеме детекције објеката, сегментацију и генерисање описа слике. Садржи више од 330,000 слика. СОСО је један од најкоришћенијих скупова података у рачунарском виду (енгл. *computer vision*, *CV*) и кориштен је за тренирање и евалуацију многих *state-of-the-art* модела за детекцију објеката и сегментацију.

За проблем генерисања описа слике користи се подскуп за тренирање, који садржи више од 118.000 слика, и подскуп за валидацију који садржи 5.000 слика. Свака слика садржи 5 описа који су ручно аотирани људи.

Слике 2.1, 2.2 и 2.3 су три случајно изабране слике из скупа за тренирање. Испод сваке слике дати су описи.

Све слике имају три димензије: висина, ширина и број канала, тако да свака слика мо е да се представи као $(h, w, 3)$ тензор. Висина и ширина слике варира,



Слика 2.1: Описи ове слике дати у скупу за тренирање су: 'A street sign on a pole on a street.', 'Two green and white street signs with building in background.', 'A city street has an intersection sign on a pole.', 'Street signs near tall buildings on the corner of Greenwich St and Vesey St.', 'A Greenwich street and Vesey street sign are hanging on a pole.'



Слика 2.2: Описи ове слике дати у скупу за тренирање су: "A white dog holding a purple frisbee in it's mouth.", 'A white dog is holding a frisbee in its mouth waiting to play.', 'White dog carrying a purple frisbee on a lawn.', "A white dog has a purple Frisbee in it's mouth.", 'a dog is holding a frisbee standing on grass'

али свака слика има 3 канала: црвени, зелени и плави (*RGB*).

Минимална висина слике је 51, максимална 640, средња 484, а медијална 480. Расподела висина слике је приказана на графику [2.4](#).

Минимална ширина слике је 59, максимална 640, средња 578, а медијална 640. Расподела ширине слике је приказана на графику [2.5](#).

Вокабулар састављен од речи које из скупа за тренирање садржи 29.077 речи.

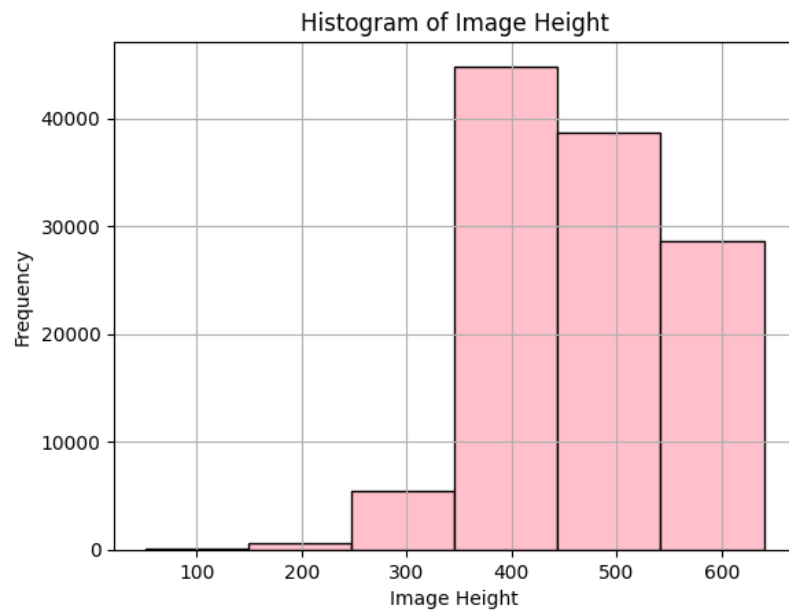
Већина слика има по 5 описа, мада има и оних са по 6 и 7 описа. Од 118.287 слика:

- 117.972 слика има по 5 описа (99.73%)
- 312 слика има по 6 описа (0.26%)
- 7 слика има по 7 описа (0.01%)



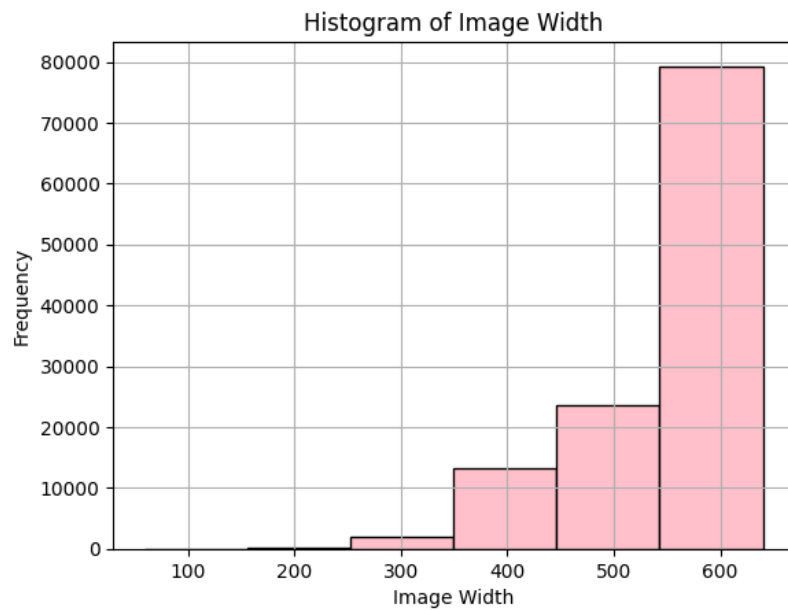
Слика 2.3: Описи ове слике дати у скупу за тренирање су: 'Woman in swim suit holding parasol on sunny day.', 'A woman posing for the camera, holding a pink, open umbrella and wearing a bright, floral, ruched bathing suit, by a life guard stand with lake, green trees, and a blue sky with a few clouds behind.', 'A woman in a floral swimsuit holds a pink umbrella.', 'A woman with an umbrella near the sea', 'A girl in a bathing suit with a pink umbrella.'

Број речи у опису варира од 5 (минимум) до 49 (максимум). Средња вредност је 10.5, а медијана је 10. Расподела броја речи у реченици је приказана на графику [2.6](#).

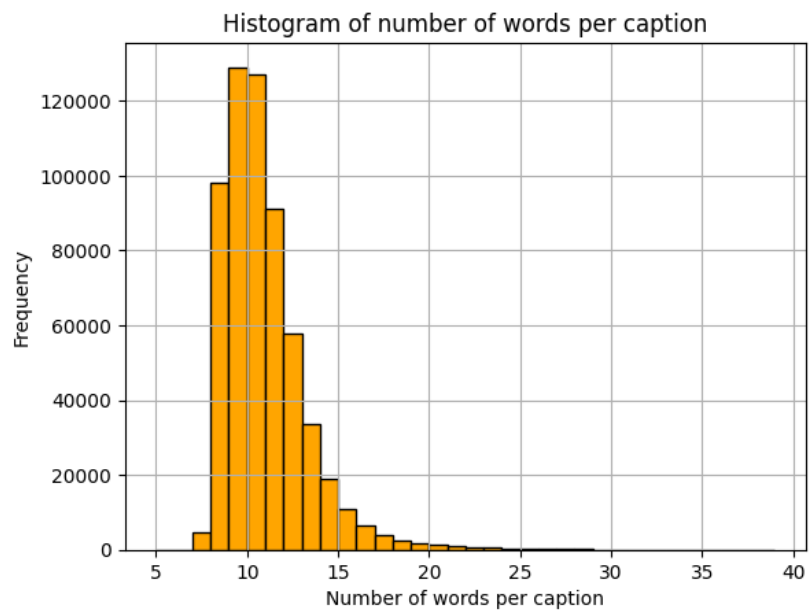


Слика 2.4: Хистограм висине слике у скупу за тренирање.

2.2 Енкодер-декодер



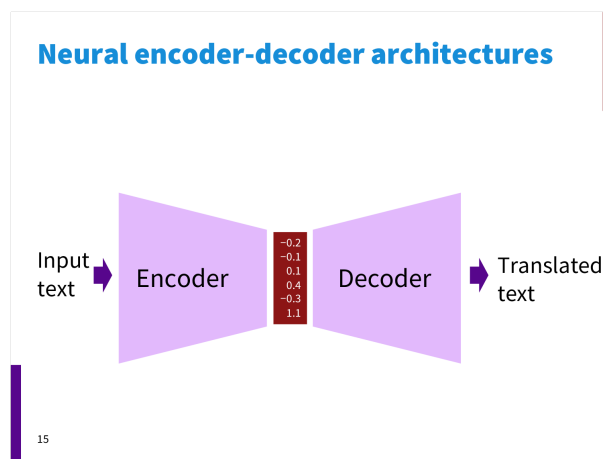
Слика 2.5: Хистограм ширине слике у скупу за тренирање.



Слика 2.6: Хистограм броја речи у опису.

2.2.1 Елементи енкодера-декодера

Енкодер-декодер је основна архитектура у парадигми машинског учења, посебно за решавање проблема пресликавања из секвенце у секвенцу (енгл. *Sequence-to-sequence*, *seq2seq*). У проблемима учења из секвенце у секвенцу, и улаз и излаз су секвенце. На пример, машинско превођење (слика 2.7) је *seq2seq* проблем јер се реченица представља као секвенца речи, улаз је реченица на једном језику и излаз је реченица на другом језику.



Слика 2.7: Модел енкодер-декодер који се користи за машинско превођење.

Архитектура енкодер-декодера састоји се од две различите неуронске мреже: енкодера и декодера.

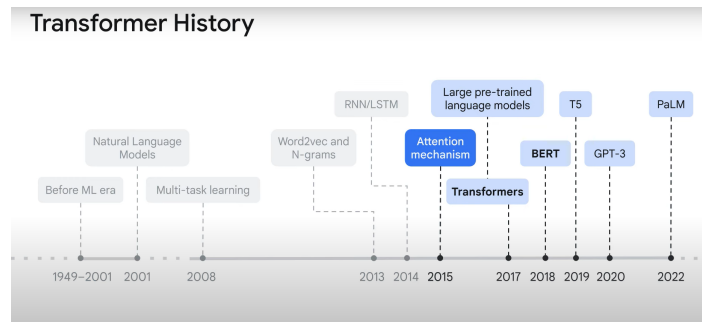
Функција енкодера је да обради улазну секвенцу (на пример, реченицу на енглеском језику). Ова обрада трансформише улазну секвенцу у вектор карактеристика (енгл. *feature vector*).

Овај вектор карактеристика (обојен црвеном бојом на слици 2.7) је вектор фиксне дужине који се назива и вектор контекста или латентни вектор. Овај вектор је дизајниран да компресује битне информације из целе улазне секвенце.

Декодер узима вектор карактеристика као свој улаз за генерисање жељене излазне секвенце. У примеру машинског превођења, ова излазна секвенца је превод реченице са енглеског на српски језик.

Скуп података за тренирање енкодера-декодера треба да се састоји од парова улаз-излаз. У задатку генерисања описа слике, скуп података за тренирање је сачињен од великог броја парова слика-опис.

И енкодер и декодер могу имати различите архитектуре, које се састоје од различитих градивних блокова који обично укључују *RNN*, *LSTM*, *CNN*, механизме пажње или трансформере, као што је приказано на слици 2.8 (Tech 2023., Не и др. 2016.).



Слика 2.8: Могући градивни блокови енкодера и декодера. Извор: Tech [2023](#).

2.2.2 Примена Енкодера-Декодера

Архитектура енкодер-декодер се такође може користити за генерисање резимеа документа, претварање говора у текст и обрнуто (енгл. *speech-to-text* и *text-to-speech*), као и описивање слика, што је тема овог рада.

Ова архитектура се може изменити и побољшати како би формирала софистицираније моделе, као што су они који садрже механизме пажње (Bahdanau, Cho и Yoshua Bengio [2016](#).), трансформере (Vaswani и др. [2023](#).), *GPT* моделе (Radford и др. [2018](#).) и *BERT* моделе (Devlin и др. [2019](#).).

Зачеци истраживања у области енкодирања и декодирања почињу 1950-их и 1960-их радом Клода Шенона и Вилбура Шрама у области теорије комуникације.

Током 1990-их и 2000-их, у домену неуронских мрежа, архитектуре енкодер-декодер је добила на значају у области машинског превођења. Структура енкодер-декодер је направила битан помак од коришћења традиционалних метода заснованих на правилима и статистичким методама ка коришћењу неуронских мрежа у машинском превођењу.

Током 2010-их, архитектура енкодер-декодер забележила је значајан пораст популарности појавом учења из секвенце у секвенце (seq2seq) (Sutskever, Vinyals и Le [2014](#).), које је користило *RNN* и *LSTM* мреже за енкодер и за декодер.

Увођење механизма пажње у раду Bahdanau, Cho и Yoshua Bengio [2016](#). године означило је још једну прекретницу за моделе "из секвенце у секвенцу" и омогућило декодеру да се селективно фокусира на најрелевантније делове улазне секвенце у сваком кораку процеса декодирања, уместо да се фокусира само на контекстни вектор фиксне величине.

Практична моћ архитектуре енкодер-декодер додатно је истакнута Гугловим редизајном своје преводилачке услуге 2016. године. Гугл преводилац је редизајниран у Гугл неуронски машински превод (енгл. *Google Neural Machine Translation*), који је користио 8-слојни двосмерни *LSTM* енкодер-декодер. Овај нови модел је значајно надмашио претходни статистички приступ, демонстрирајући ефикасност *seq2seq* учења у пракси.

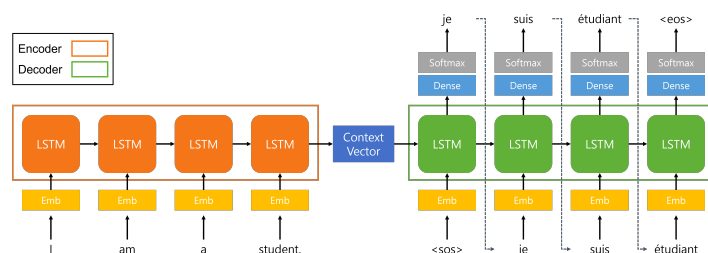
Архитектура трансформера, коју је представио (Vaswani и др. [2023](#).) у свом револуционарном раду, додатно је поставила архитектуру енкодер-декодер као стандард у обради природног језика (енгл. *Natural Language Processing, NLP*).

До 2015. године, *NLP* модели су у великој мери прешли са статистичких на моделе базиране на неуронским мрежама, при чему је архитектура енкодер-

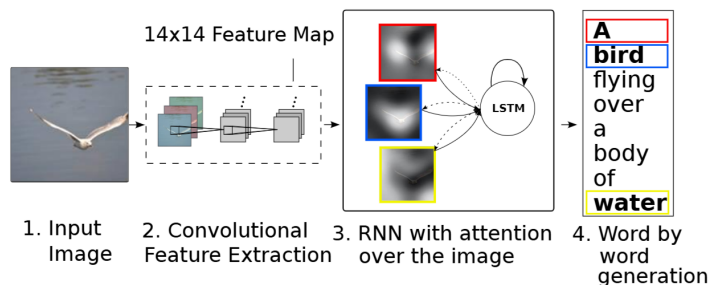
-декодер играла централну улогу.

2.2.3 Градивни блокови енкодера и декодера

И енкодер и декодер могу имати различите архитектуре неуронских мрежа, које се састоје од различитих градивних блокова који обично укључују рекурентну неуронску мрежу, *LSTM*, конволуциону неуронску мрежу (енгл. *Convolutional Neural Network, CNN*), механизме пажње или трансформер. На слици 2.9 је приказан модел енкодер-декодер који користи *LSTM* блок и у енкодеру и у декодеру, а на слици 2.10 је приказан модел енкодер-декодер који користи *CNN* блокове у енкодеру и механизам пажње и *RNN* мрежу у декодеру.



Слика 2.9: Модел енкодер-декодер који користи *LSTM* блок и у енкодеру и у декодеру. Извор: (Singh 2023.)



Слика 2.10: Модел енкодер-декодер који користи *CNN* блок у енкодеру и механизам пажње и *RNN* мрежу у декодеру. Извор: (Ху и др. 2015.)

Скуп података $D = \{x_n, y_n\}_{n=1}^N$ за тренирање енкодера-декодера се састоји од парова улаз-излаз које модел треба да научи. У примеру машинског превођења, скуп података за тренирање се састоји од многобројних парова улаз-излаз, где се пар са индексом n састоји од низа речи x_n и одговарајућег низа речи превода y_n .

Пример скупа података за тренирање енкодера-декодера

Скуп података за задатак машинског превођења састоји се од реченица на српском и одговарајућих превода на енглески језик:

1. "Danas je petak" → "Today is Friday"

2. "Lepo je vreme" → "It's a nice weather"

Ако се српски речник састоји од речи ['данас', 'је', 'петак', 'лепо', 'време'], one-hot енцирани вектор који представља реч 'петак' је $[0, 0, 1, 0, 0]^T$. Слично томе, енглески речник је онда ['today', 'is', 'friday', 'it's', 'a', 'nice', 'weather']. Скуп података се састоји од следећих секвенци:

$$\begin{aligned} x_1 &= [[1, 0, 0, 0, 0], [0, 1, 0, 0, 0], [0, 0, 1, 0, 0]] \rightarrow \text{"Danas je petak"} \\ y_1 &= [[1, 0, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0]] \rightarrow \text{"Today is Friday"} \\ x_2 &= [[0, 0, 0, 1, 0], [0, 1, 0, 0, 0], [0, 0, 0, 0, 1]] \rightarrow \text{"Lepo je vreme"} \\ y_2 &= [[0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], \\ &\quad [0, 0, 0, 0, 0, 1, 0], [0, 0, 0, 0, 0, 0, 1]] \rightarrow \text{"It's a nice weather"} \end{aligned}$$

2.2.4 Тренирање енкодер-декодер модела

Енкодер пресликава улазни вектор x_n у контекстни вектор фиксне дужине $z_n = \text{Encoder}(x_n)$, а затим декодер узима контекстни вектор z_n као свој улаз и пресликава га у излази низ $\hat{p}_n = \text{Decoder}(c_n)$. Највиши слој декодера користи *softmax* активациону функцију, претварајући излаз у расподелу вероватноће, и због тога излаз декодера посматрамо као низ случајних вектора.

У примеру машинског превођења, $\hat{p}_n$ је предикција модела за превод. Y_n је низ случајних вектора јер је излаз декодера расподела вероватноће по целом речнику.

Параметари енкодера и декодера који се уче (тежине, помераји...) се конкатенирају у један вектор који означавамо θ . Циљ тренирања модела је проналажење оптималне вредности θ које максимизирају функцију веродостојности исправне излазне секвенце на основу улазне секвенце:

$$\begin{aligned} \theta^* &= \arg \max_{\theta} L(\theta; D) \\ &= \arg \max_{\theta} P_{\theta}\{Y_1 = y_1, \dots, Y_N = y_N | x_1, \dots, x_N\} \\ &\stackrel{\text{independence}}{=} \arg \max_{\theta} \prod_{n=1}^N P_{\theta}\{Y_n | x_n\} \end{aligned}$$

Максимизирање логаритамске функције веродостојности је математички еквивалентно максимизирању саме функције веродостојности јер је логаритам монотонно растућа функција:

$$\begin{aligned} \theta^* &= \ln L(\theta; D) \\ &= \arg \max_{\theta} \ln \prod_{n=1}^N P_{\theta}\{Y_n = y_n | x_n\} \\ &= \arg \max_{\theta} \sum_{n=1}^N \ln P_{\theta}\{Y_n = y_n | x_n\} \end{aligned}$$

И скуп улазних и скуп излазних секвенци се састоје од секвенци које су различитих дужина. Из практичних разлога, допуњавамо улазне и излазне низове до најдужега низа у скупу за тренирање L_1 и L_2 , обично са нула-вектора. Након допуне, сваки улазни низ има облик матрице (вектор вектора) $x_n = [x_{n,1}, x_{n,2}, \dots, x_{n,L_1}]$ и сваки излаз има сличан облик $y_n = [y_{n,1}, y_{n,2}, \dots, y_{n,L_2}]$.

Случајна матрица Y_n (тј. њена функција расподеле) описује предикцију модела $\text{Decoder}(\text{Encoder}(x_n))$.

У проблему машинског превођења, $y_{n,1}$ је прва реч n -тог узорка која је "one-hot" кодирана, а $Y_{n,1}$ је вектор који представља предикцију модела. $\hat{p}_{n,v}$ је предвиђена расподела по речима речника.

$$\begin{aligned}\theta^* &= L(\theta; D) \\ &= \arg \max_{\theta} \sum_{n=1}^N \ln P_{\theta}\{Y_n = y_n | x_n\} \\ &= \arg \max_{\theta} \sum_{n=1}^N \ln P_{\theta}\{Y_{n,1} = y_{n,1}, \dots, Y_{n,L_2} = y_{n,L_2} | x_n\}\end{aligned}$$

Пошто за декодер користимо рекурентну неуронску мрежу (RNN), реч у кораку t зависи од свих претходних предвиђања у корацима $1, 2, \dots, t-1$. Користећи Бајесову теорему, имамо следеће:

$$\begin{aligned}\theta^* &= \arg \max_{\theta} L(\theta; D) \\ &= \arg \max_{\theta} \sum_{n=1}^N \ln P_{\theta}\{Y_{n,1} = y_{n,1}, \dots, Y_{n,L_2} = y_{n,L_2} | x_n\} \\ &\stackrel{\text{Бајесова теорема}}{=} \arg \max_{\theta} \sum_{n=1}^N \sum_{t=1}^{L_2} \ln P_{\theta}\{Y_{n,t} = y_{n,t} | x_n, Y_{n,1} = y_{n,1} \dots Y_{n,t-1} = y_{n,t-1}\}\end{aligned}$$

Оптимизациони проблем максимизације веродостојности у односу на параметар θ , за дат скуп података, еквивалентан је проблему минимизације суме крос ентропија (енгл. *cross-entropy loss*) $L(y_{n,t}, \hat{p}_{n,t})$ између анотација $y_{n,t}$ и предвиђања декодера $\hat{p}_{n,t} \stackrel{\text{def.}}{=} P_{\theta}\{Y_{n,t} = y_{n,t} | x_n, Y_{n,1} = y_{n,1} \dots Y_{n,t-1} = y_{n,t-1}\}$, где сума обухвата сваки од узорака n и све њихове временске кораке t . Ова сума је детаљније објашњена у [2.4.2](#).

$$\begin{aligned}\theta^* &= \arg \max_{\theta} \sum_{n=1}^N \sum_{t=1}^{L_2} \ln P_{\theta}\{\hat{Y}_{n,t} = y_{n,t} | x_n, \hat{Y}_{n,1} = y_{n,1} \dots \hat{Y}_{n,t-1} = y_{n,t-1}\} \\ &= \arg \min_{\theta} \sum_{n=1}^N \sum_{t=1}^T L(y_{n,t}, \hat{p}_{n,t})\end{aligned} \tag{2.1}$$

Крос ентропија (cross-entropy loss) се дефинише као:

$$L(y_{n,t}, \hat{p}_{n,t}) \stackrel{def.}{=} - \sum_{l=1}^{L_2} y_{n,t,l} \ln \hat{p}_{n,t,l}$$

Коначан циљ тренирања је проналзак оптималних параметара θ^* који минимизују функцију цене дефинисану у 2.1. За овај оптимизациони проблем се у пракси користи градијентни спуст (енгл. *gradient descent*).

Током процеса тренирања, параметри модела који се тренирају (енгл. *trainable parameters*) (тежине и помераји у оквиру и енкодера и декодера) — итеративно се подешавају помоћу алгоритма оптимизације заснованог на градијентном спусту, уз помоћ пропагације сигнала грешке уназад.

2.3 ResNet

2.3.1 Историјска позадина

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) Russakovsky и др. 2015. је међународно такмичење у рачунарском виду које се одржавало сваке године од 2010. до 2017. године. Организовано је од стране универзитета *Stanford* и *Princeton*, а заснивало се на огромном скупу података *ImageNet*, који садржи преко 14 милиона слика подељених у 1000 категорија (нпр. животиње, предмети, возила...). Такмичари су имали задатак да направе алгоритам који што тачније:

- класификује слике,
- локализује или детектује објекте унутар слика.

Такмичење је било кључно за развој рачунарског вида и дубоког учења. Након 2017. ILSVRC више није званично организован, јер су модели достигли готово људску тачност.

Основе *CNN*-ова су постављене и популаризоване још 1980-их година радом Јана Лекуна (Yann LeCun) и сарадника (LeCun, Boser и др. 1989.; LeCun, Bottou и др. 2002.) на моделима *LeNet-1* и *LeNet-5* за препознавање руком писаних цифара, али тек појава великог скупа података *ImageNet* и рачунарске снаге савремених *GPU*-ова омогућила је њихову пуну примену.

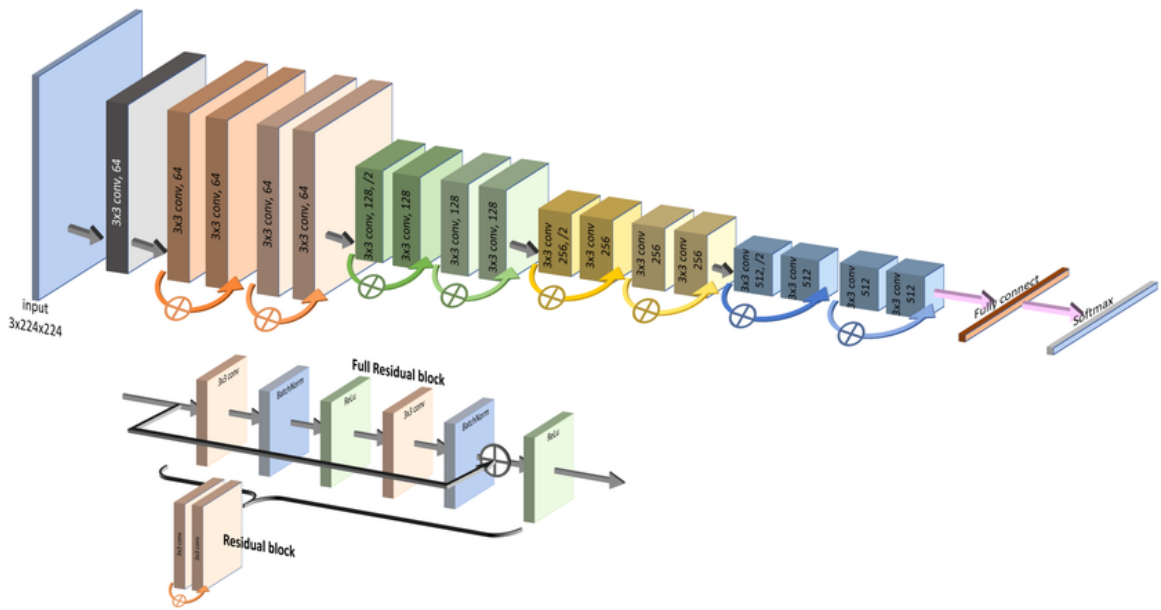
Пре 2012. године, најуспешнији модели на *ImageNet* такмичењу користили су класичне методе машинског учења засноване на ручно задатим карактеристикама и филтерима (енгл. *handcrafted features*), након чега ти подаци прослеђивани класификаторима попут *SVM*-а (*Support Vector Machine*). Иако су ови модели постигали релативно добре резултате (са грешком од 25%), њихова способност генерализације и скалирања на велике и разноврсне скупове попут *ImageNet*-а била је ограничена. Ово је трајало до револуционарне 2012. године.

Модел *AlexNet* (Krizhevsky, Sutskever и Hinton 2012.) је убедљиво победио на *ImageNet* такмичењу 2012. године са грешком од 16%. *AlexNet* је био први дубоки *CNN* трениран на великом скупу података (1.2 милиона слика) уз употребу *GPU*-а, што је омогућило практичан тренинг дубоке мреже са осам слојева. *AlexNet* је означио почетак ере дубоких неуронских мрежа и вратио *CNN* у центар пажње научне заједнице.

У наредним годинама модели постају дубљи, са 20-ак слојева и користе мање филтере различитих величина (Simonyan и Zisserman 2014.; Szegedy и др. 2015.).

2015. године на такмичењу побеђује ResNet архитектура (He и др. 2016.) са грешком од 4%. *ResNet* је решио проблем опадања перформанси дубоких мрежа са порастом броја слојева увођењем резидуалних веза (енгл. *residual (skip) connections*). Резидуалне везе омогућавају тренирање мреже са неколико стотина слојева. ResNet и данас представља стандард у дубоком учењу и рачунарском виду.

Да бисмо разумели *ResNet* архитектуру, илустровану на 2.11, потребно је да разумемо градивне јединице овог модела: конволуциони блок и резидуални блок, те су ови подмодели објашњени у наредним поглављима.



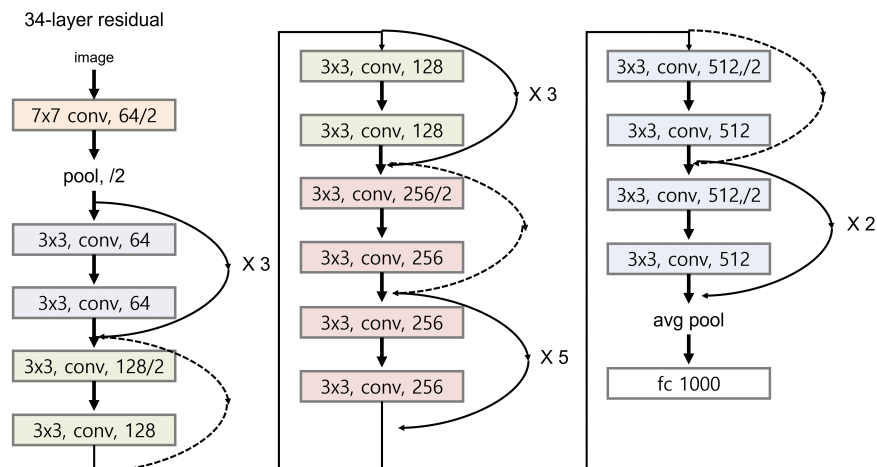
Слика 2.11: Архитектура *ResNet* модела (илустрација изнад). *ResNet* се састоји од неколико коволуционих блокова, сваки са резидуалном везом. Један конволуциони блок са резидуалном везом чини резидуални блок (илустрација испод). Извор: Jinning 2022.

2.3.2 Конволуциони блок

Конволуционални блок (енгл. *Convolutional Block*) је неуронска мрежа која на улазу узима слику и "извучи" из ње значајне карактеристике које избацује на излазу. Излаз из конволуционог блока су **мапе карактеристика** (енгл. "**feature maps**"). Конволуциони блок је фраза која се користи за низ слојева унутар **конволуционе неуронске мреже** (енгл. *Convolutional Neural Network, CNN*), као што је *ResNet*. Конволуциони блокови се често понављају јер се конволуциона неуронска мрежа добија слагањем неколико различитих типова конволуционих блокова, што је илустровано на 2.11.

Конволуциони блок се састоји од **конволуционих слојева**. Један слој на илустрацији 2.11 је означен као један квадар. Приметимо да сваки конволуциони слој мења просторну димензију (висину и ширину) и дубинску димензију улаза. Улаз у *ResNet* је $3 \times 224 \times 224$ слика која има просторну димензију 224×224 и дубину 3. Каже се и да улазна слика има 3 канала. Након сваког конволуционог блока, на илустрацији означеног сивом стрелицом, просторна димензија и број канала мапа карактеристика се мења. *ResNet* је дизајниран тако да се просторна димензија се смањује, а број канала расте, како улазна слика пролази кроз мрежу. Конволуциони блок се састоји од једног или више конволуционих слојева које прати "*pooling*" слој.

Други начин да се визуелизира *ResNet* модел је приказан на 2.12.



Слика 2.12: Архитектура *ResNet 34* модела. Један конволуциони слој је представљен као један правоугаоник. Један или више конволуционих слојева чине конволуциони блок. Један конволуциони блок чине конволуциони слојеви обојени истом бојом. Извор: Kurnarun и др. 2023.

Конволуциони слој

На слици 2.13 је приказан један **конволуциони слој** чији је улаз $3 \times 224 \times 224$ слика, а излаз $1 \times 224 \times 224$ мапа карактеристика. Приказан конволуциони слој чини један 3×3 **филтер** са 3 канала (кернела) и **активациона функција**. Филтер се "креће" по улазној слици и на свакој позицији рачуна конволуцију. Резултат једне конволуције је један број који пролази кроз активациону функцију и уписује се у мапу карактеристика. Мапа карактеристика представља преглед карактеристика улаза.

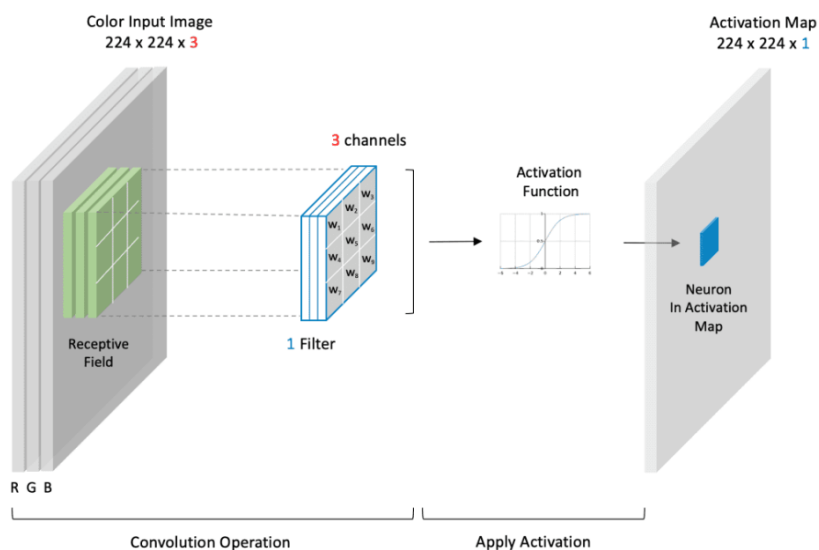
Конволуциони слој обично садржи више филтера. Да конволуциони слој на слици 2.13 садржи N филтера, излаз би била мапа карактеристика димензија $N \times 224 \times 224$. Дакле, број филтера слоја одређује дубину излазне мапе карактеристика.

Операција конволуције

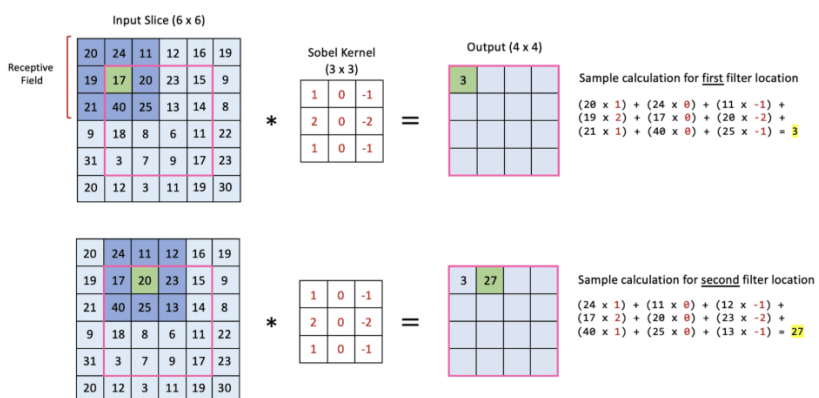
Операција конволуције састоји се од постављања кернела преко дела улаза и поелементног множења кернела са одговарајућим елементима улаза и сабирања резултата. Добијена вредност је један број који представља излаз операције конволуције за одређену локацију филтера. Процес се понавља превлачењем филтера преко улазне слике све док филтер није постављен преко сваког дела улаза. Прва два корака су приказана на 2.14.

Stride одређује дужину корака, тј. колико пиксела се кернел шифтује у једном тренутку. Смањење *stride*-а резултује учењем више карактеристика и већом просторном димензијом излаза.

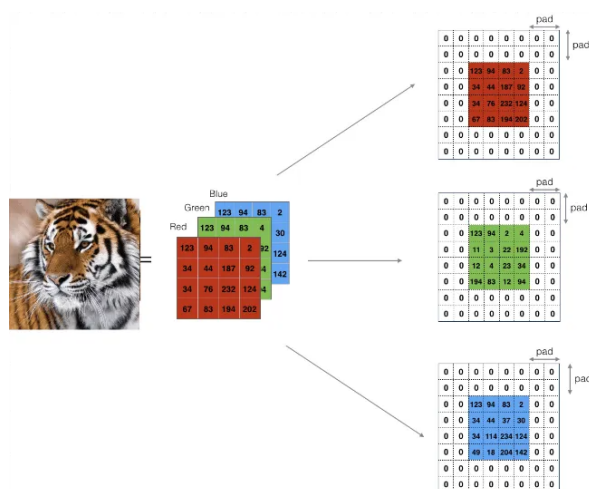
Padding је важан параметар у *CNN*-у јер помаже у очувању просторне димензије улаза додавањем додатних пиксела око граница улазне слике као што је приказано на 2.15. На тај начин се у излазу чувају информације о карактеристикама на ивицама.



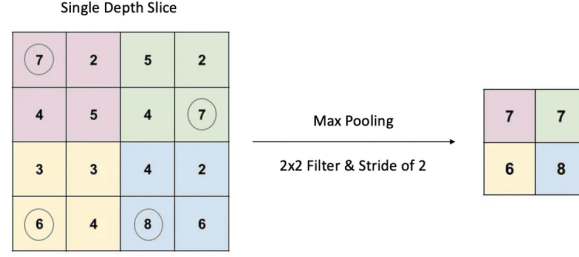
Слика 2.13: Конволуциони слој чији је улаз $3 \times 224 \times 224$ слика, а излаз $1 \times 224 \times 224$ мапа карактеристика. Приказан конволуциони слој чини један 3×3 филтер са 3 канала (кернала) и активациона функција. Извор: Kromydas 2023.



Слика 2.14: Операција конволуције. Извор: Kromydas 2023.



Слика 2.15: Улазnoj слици је додат **padding** дужине 2. Извор: Kromydas 2023.



Слика 2.16: Илустрација *max pooling* слоја. Извор: Kromydas 2023.

Димензије излаза из конволуционог слоја

Нека је улаз у конволуциони слој слика димензија $c \times w \times h$. Применимо конволуциони слој са n филтера $k_1 \times k_2$, страјдом дужине s , где је $padding = p$. Димензије излазне мапе карактеристика су $n \times m_1 \times m_2$, где је:

$$m_1 = \left\lfloor \frac{h + 2p - k_1}{s} \right\rfloor$$

$$m_2 = \left\lfloor \frac{w + 2p - k_2}{s} \right\rfloor,$$

Код конволуционог слоја сви кернели свих филтера су параметри који се уче током тренирања.

2.3.3 Max Pooling слој

У *CNN* архитектурама се често користи *max pooling* слој након неколико конволуционих слојева. *max pooling* слој смањује просторне димензије улаза. Коришћење овог слоја смањује број параметара у мрежи јер је улазна величина у следећим слојевима смањена. Такође, коришћење мање параметара често помаже у смањењу ефеката "учења напамет" (енгл. *overfitting*).

max pooling слој са 2×2 филтером и страјдом једнаким 2 је илустрован на 2.16.

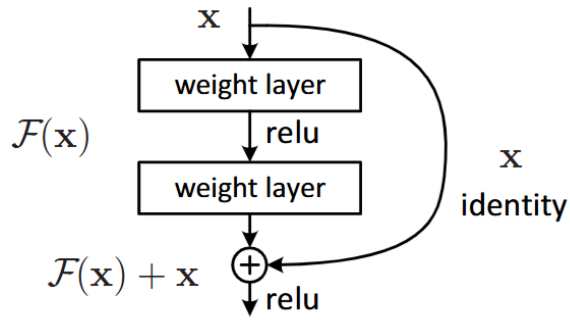
Нека је улаз у *max pooling* слој мапа карактеристике димензија $c \times w \times h$. Применимо *max pooling* слој са $p_1 \times p_2$ филтером и страјдом дужине s . Димензије излазне мапе карактеристика су $c \times (\lfloor \frac{h-p_1}{s} \rfloor + 1) \times (\lfloor \frac{w-p_2}{s} \rfloor + 1)$.

Max pooling слој нема ни један параметар који се учи током тренирања.

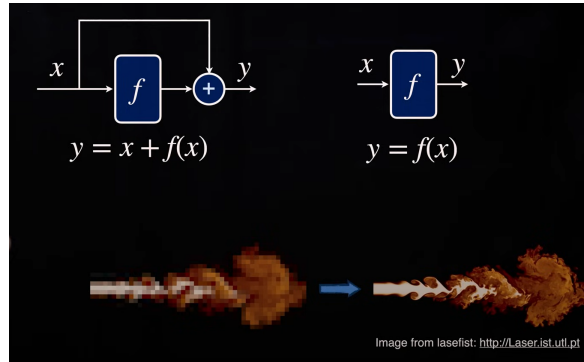
2.3.4 Резидуални блок

Градивна јединица *ResNet*-а је резидуални блок (енгл. *residual (skip) block*, приказана на слици 2.17. Резидуални блок је сам за себе неуронска мрежа.

Идеја иза резидуалног блока је приказана на слици 2.18. Са H означимо жељено пресликавање. Уместо да мрежа од почетка учи да апроксимира функцију, мрежа (резидуални блок) учи да апроксима "само" резидуалну функцију F , где је $F(x) = H(x) - x$, слагањем неколико нелинеарних слојева. Оригинални рад (He и др. 2016.) износи хипотезу а је једноставније апроксимирати пресликавање F него пресликавање H . Наводи се да, у екстремном случају, уколико је



Слика 2.17: Илустрација резидуалног блока (енгл. *residual (skip) block*, где x представља улаз у блок, F је неуронска мрежа са два слоја чије се матрице тежине и пристрасности уче и $y = x + F(x)$ је излаз из блока. Извор: Не и др. 2016.



Слика 2.18: Извор: Brunton 2024.

H идентичка функција, једноставније је да мрежа "научи" нула пресликавање него идентичко пресликавање слагањем нелинеарних слојева.

Forward pass резидуалног блока је дефинисан са:

$$y = F(x) + x, \quad (2.2)$$

где је x улаз у блок, F је неуронска мрежа која настаје слагањем нелинеарних слојева, а y је предвиђање блока.

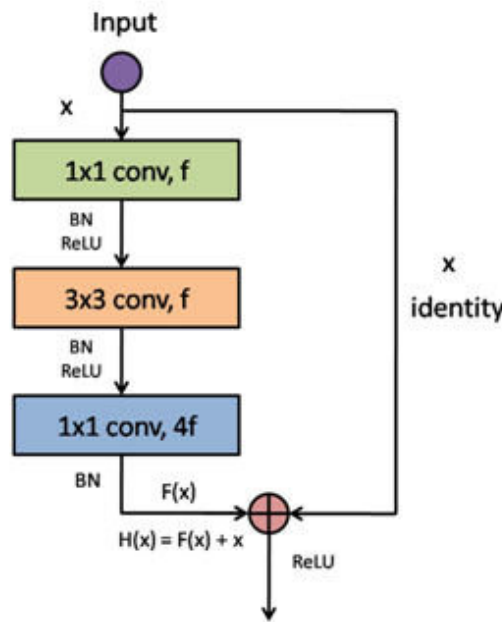
На слици 2.17 неуронску мрежу резидуалног блока чине два линеарна слоја са *ReLU* активационом функцијом:

$$y = (W_2 \cdot \text{ReLU}(W_1 \cdot x + b_1) + b_2) + x, \quad (2.3)$$

где су W_1 и W_2 матрице тежина, а b_1 и b_2 пристрасност линеарних слојева.

Дефинишимо *shortcut* везу као додавање улаза x на крај блока, приказаног у формулама 2.2 и 2.3. Додавање *shortcut* везе не повећава број параметра за учење у моделу, те оне успоравају учење градијентним спустом.

Нелинеарни слојеви у резидуалним блоковима чијим слагањем настаје функција F могу бити и конволуционе неуронске мреже. Баш то је случај и код *ResNet* модела што је детаљније објашњено у наредном поглављу.



Слика 2.19: *Bottleneck Block* у *ResNet-152* моделу

2.3.5 *Batch Normalization* слој

Batch Normalization који се користи у *CNN*-овима да би се нормализовао излаз претходног активационог слоја одузимањем средње вредности *batch*-а и дељењем са стандардном девијацијом *batch*-а, након чега следи скалирање и померање вредностима које се уче. Овај процес убрзава тренирање и уводи благи шум у мрежу ради боље генерализације.

2.3.6 *Bottleneck* блок у *ResNet-152*

Bottleneck блок, илустрован на 2.19, је градивна јединица *ResNet-152* модела која примењује концепт резидуалног учења и конволуционе неуронске мреже.

Сваки *bottleneck* блок се састоји од 3 конволуциона слоја са димензијама керна 1×1 , 3×3 и 1×1 . Сваки конволуциони слој прати *batch normalization* слој и активациони слој са *ReLU* активационом функцијом, осим последњег конволуционог слоја који нема пратећи *ReLU* активациони слој. Поменуте слојеве прати резидуална веза.

2.3.7 Архитектура *ResNet* модела

ResNet-152 consists of an initial convolutional layer and four main stages of bottleneck blocks with increasing channel dimensions. Each stage contains a different number of bottleneck blocks—3, 8, 36, and 3 respectively—yielding a total of 152 layers (1 initial convolution + 50 bottleneck blocks $\times$ 3 layers per block + final fully connected layer). Thus, the bottleneck block serves as the principal structural unit

that enables ResNet-152 to achieve high representational power while maintaining computational tractability.

ResNet модел садржи **пет фази** и **линеарног слоја са *softmax* активационом функцијом**. *ResNet* модел је трениран на *ImageNet* скупу података, који садржи 1,000 класа и због тога излазни линеарни слој има 1,000 неурона на излазу чије вредности, након *softmax* активације, представљају вероватноће да улазна слика припада свакој класи.

Прва фаза почиње конволуционим слојем димензијама кернела 7×7 и 64 филтера, *stride*-ом једнаким 2 и *padding*-ом једнаким 3. Овај слој бележи визуелне карактеристике најнижег нивоа као што су ивице и углови. Конволуциони слој прати *batch normalization* слој, активациони слој са *ReLU* активационом функцијом и *max pooling* 3×3 слој са *textitstride*-ом једнаким 2.

Прва фаза трансформише сирову улазну слику $3 \times 224 \times 224$ у скуп мапа карактеристика димензије $64 \times 56 \times 56$.

У **другој фази** почиње резидуално учење, уводећи прву групу *bottleneck* блокова. Ова фаза има 3 *bottleneck* блока. Излазна мапа карактеристика из Фазе 2 има димензије $256 \times 56 \times 56$ и садржи карактеристике средњег нивоа, као што су делови објекта и контуре.

Трећа фаза продубљује мрежу слагањем осам *bottleneck* блокова, од којих сваки производи 512 канала. Излазна мапа карактеристика након 3. фазе је димензија $28 \times 28 \times 512$ и садржи сложеније просторне и семантичке карактеристике, укључујући облике објеката и грубе просторне односе.

Фаза 4 представља најдубљи и рачунарски најинтензивнији део *ResNet-152* модела. Она се састоји од 36 *bottleneck* блокова, од којих сваки производи 1024 канала. Добијене мапе карактеристика су димензија $1024 \times 14 \times 14$ и садрже контекстуалне информације средњег и високог нивоа неопходне за категоризацију и локализацију објеката.

Пета (последња) фаза састоји се од 3 *bottleneck* блока, од којих сваки производи 2048 излазних канала. По завршетку 5. фазе, резултујуће мапе карактеристика су димензије $2048 \times 7 \times 7$, који се затим пропушта кроз финални линеарни слој.

2.4 LSTM

LSTM је посебна врста неуронске мреже која се користи за *seq2seq* учење и складиштење интерне меморије током дужег времена.

Ова секција ће покрити:

- Моделирање секвенци у поглављу [2.4.1](#)
- Ванила рекурентна неуронска мрежа у поглављу [2.4.2](#)
- *LSTM* у поглављу [2.4.3](#)

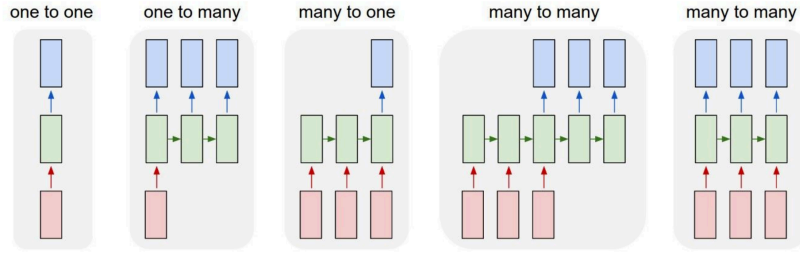
2.4.1 Моделирање секвенци

Неуронске мреже са директним простирањем сигнала (енгл. *feedforward neural networks*) су најједноставнији тип неуронске мреже, где су улаз и излаз вектори фиксне величине $x \in R^D$, попут слике димензије $224 \times 224 \times 3$, као што је илустровано на слици [2.20](#) скроз лево. Неуронске мреже са директним простирањем сигнала нису дизајниране да као улазне податке обрађују **секвенцијалне податке** $(x_1, x_2, \dots, x_T) \in R^D$ као што су реченице или временски низови. Рекурентне неуронске мреже (RNN) (Rumelhart, Hinton и Williams [1986](#).) су фамилија неуронских мрежа за обраду секвенцијалних података. Рекурентне неуронске мреже могу имати неку од архитектуре приказаних на графиконима [2.20](#):

- **секвенца као излаз** - нпр. генерисање описа слика - на улазу је слика, вектор фиксне величине, а на излазу је секвенца речи променљиве дужине.
- **секвенца као улаз** - нпр. анализа сентимената (енгл. *sentiment analysis*), у којој се дата реченица (низ речи, које су представљене као вектор) класификује на основу тога да ли изражава позитивно или негативно расположење.
- **секвенца и као улаз и као излаз** - нпр. машинско превођење, у којем рекурентна неуронска мрежа (RNN) добија реченицу на енглеском као улаз, а модел треба да генерише реченицу на српском.
- **синхронизовани секвенцијални улаз и излаз** - класификација видеа где излаз означава лабелу сваког видео кадра.

Користићемо последњу архитектуру модела ("many-to-many") да формализујемо нотацију моделирања секвенци. И улаз и излаз су секвенце исте дужине T . Улазна секвенца је означена као $x_1, x_2, \dots, x_T \in R^D$, а излазна секвенца је означена као $y_1, y_2, \dots, y_T \in R^K$. У задатку класификације фрејмова видео записа, ако је видео кадар слика величине $(224, 224, 3)$, онда је улазни вектор x_t вектор фиксне величине $x_t \in R^D$, $D = 224 \cdot 224 \cdot 3$ у сваком временском кораку t . Ако сваки видео кадар треба да буде означен као једна од K ознака, онда је ознака $y_t \in R^K$ вектор "one-hot" кодиран.

RNN моделира вероватноћу да улаз x_t има лабелу $k \in \{1, \dots, K\}$, користећи информације о претходним улазима $x_1, \dots, x_{t-1}$. Нека је Y_t случајни вектор који



Слика 2.20: Различите врсте архитектура RNN модела. Прва лево: неуронска мрежа са директним простирањем сигнала. Црвени правоугаоници су улазни вектори. Зелени правоугаоници су скривени слојеви. Плави правоугаоници су излазни вектори. Извор: Johnson 2021.

описује лабелу у временском кораку t . $\hat{Y}_t$ може имати K различитих вредности, све "one-hot" кодирани вектори, који описују K различитих лабела:

$$\Omega_{Y_t} = \underbrace{\{[1, 0, \dots, 0]^T\}}_{\text{прва класа}}, \underbrace{\{[0, 1, \dots, 0]^T\}}_{\text{друга класа}}, \dots, \underbrace{\{[0, 0, \dots, 1]^T\}}_{K\text{-та класа}}$$

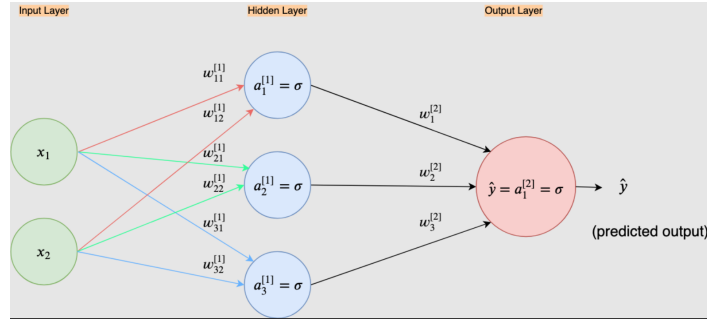
Израз RNN модела су вероватноће $\hat{p}_{t,1}, \hat{p}_{t,2}, \dots, \hat{p}_{t,K}$ које чине расподелу случајног вектора Y_t :

$$P(Y_{t,k} = 1) = \begin{cases} \hat{p}_{t,1}, & \text{if } k = 1 \\ \hat{p}_{t,2}, & \text{if } k = 2 \\ \dots & \\ \hat{p}_{t,K}, & \text{if } k = K \end{cases} \quad (2.4)$$

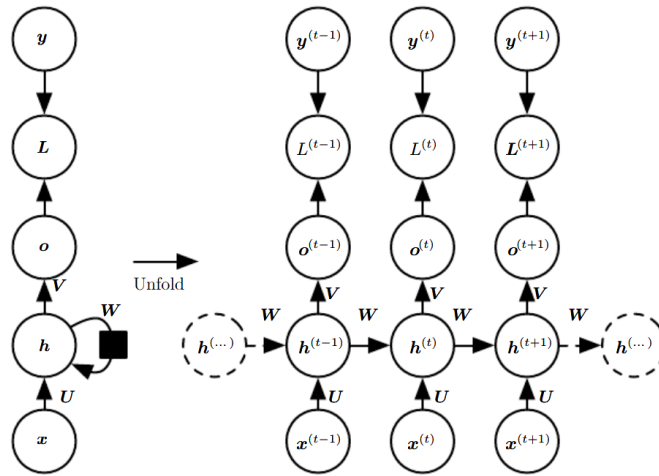
Рекурентне неуронске мреже користе једну од раних идеја из машинског учења и статистичких модела из 1980-тих: **дељење параметара** између различитих делова модела. Делење параметара омогућава примену модела на примере различитих дужина и генерализацију над њима (Goodfellow, Yoshua Bengio и Courville 2016.).

Графови израчунавања (енгл. *computational graphs*) у неуронским мрежама са директним простирањем сигнала немају циклусе, као што је приказано на слици 2.21. Графови израчунавања RNN-ова, с друге стране, могу да имају циклусе, као што је приказано на слици 2.22. Циклуси представљају утицај садашње вредности променљиве на њену вредност у наредном временском кораку.

„Ванила“ рекурентне неуронске мреже су најједноставнији тип рекурентних неуронских мрежа. У наредним подсекцијама укратко објашњавамо како функционишу и разлоге због којих нису толико ефективне у пракси.



Слика 2.21: Граф израчунавања неуронске мреже са директним простирањем сигнала. Нема циклуса. Извор: Тапо 2021.



Слика 2.22: Граф израчунавања рекурентне неуронске мреже. Циклуси су присутни. Лево: *RNN* и њена функција цене су нацртани са рекурентним везама. Десно: Исти модел приказан графом израчунавања развијеном у времену, где је сваки чвор повезан са једним одређеним временским тренутком. Извор: Goodfellow, Yoshua Bengio и Courville 2016.

2.4.2 Vanilla Recurrent Neural Network

Forward Pass

Ванила *RNN* има „унутрашње стање“ (енгл. *internal state*) које се назива скривено стање (енгл. *hidden state*) и које се ажурира како се секвенца обрађује. Скривено стање у временском кораку t је вектор, означен као h_t , који чува информације о тренутном улазу x_t и свим претходним улазима $x_1, x_2, \dots, x_{t-1}$. Тренутно скривено стање h_t је рекурентно добијено коришћењем претходног скривеног стања h_{t-1} и тренутног улаза x_t , једначином $h_t = f(h_{t-1}, x_t)$. Функција f у ванила *RNN* је једноставан линеарни слој са $\tanh$ активационом функцијом дефинисаном у 2.5 и илустрованом у 2.22.

$$h_t = \tanh(W \cdot h_{t-1} + U \cdot x_t + b) \quad (2.5)$$

Ако је наш задатак обележавање сваког видео фрејма, скривено стање у тренутку t ће чувати информације о сценама и глумцима у претходним фрејмовима. Излаз у тренутку t , означен са o_t , је функција h скривеног стања у истом тренутку $t = h(h_t)$. Функција h у ванила RNN -овима је једноставан линеарни слој дефинисан у 2.6 и илустрован у 2.22.

$$o_t = V \cdot h_{t-1} + c \quad (2.6)$$

Функција активације примењена на излаз o_t зависи од проблема који RNN решава. На пример, за проблем лабелирања видеа, у којем сваки фрејм (енгл. *frame*) припада некој од K категорија, се користи *softmax* активациона функција као што је дефинисано у 2.7, на тај начин чинећи расподелу вероватноће $\hat{p}_t$ мреже по свим могућим лабелама.

$$\hat{p}_t = \text{softmax}(o_t) = \text{softmax}(V \cdot h_{t-1} + c) \quad (2.7)$$

Постоји K различитих класа, тако да $\hat{p}_t$ има облик K -димензионалног вектора $\hat{p}_t = [\hat{p}_{t,1}, \hat{p}_{t,2}, \dots, \hat{p}_{t,K}]^T$. Излаз $\hat{p}_{t,k}$ посматрамо као RNN -ову предикцију вероватноће да Y_t припада класи k у временском кораку t описано једначином 2.8.

$$\hat{p}_{t,k} := P_{RNN}\{Y_t = \underbrace{[0, \dots, 0, 1, 0, \dots, 0]^T}_{\text{1 is on } k\text{-th place}} | x_1, x_2, \dots, x_{t-1}\}, \quad \forall k \in \{1, 2, \dots, K\} \quad (2.8)$$

Једначине 2.5, 2.6 и 2.7 дефинишу *forward pass* улаза кроз ванила RNN модел.

Тренирање Ванила RNN модела

Параметри који се уче (енгл. *learnable parameters*) у ванила RNN моделу су матрице тежина W, U и V , и помераји b и c . Ради лакше нотације, сви параметри који се могу научити су конкатенирани у један вектор и означени као параметар θ .

Циљ тренирања мреже је проналажење параметара θ^* који максимизирају веродостојност скупа података за тренирање величине N : $D = \{(x_1^n, \dots, x_T^n), (y_1^n, \dots, y_T^n)\}_{n=1}^N$.

$$\theta^* = \arg \max_{\theta} P_{\theta}\{Y = y|x\} \quad (2.9)$$

$$= \arg \max_{\theta} P_{\theta}\{Y_1 = y_1, \dots, Y_N = y_N | x_1, \dots, x_N\} \quad (2.10)$$

С обзиром на то да су узорци у скупу података независни и случајни вектор $Y_{n,t}$ у временском кораку t зависи само од претходних улаза $x_{n,1}, x_{n,2}, \dots, x_{n,t-1}$

n -тог узорка, имамо:

$$\theta^* = \arg \max_{\theta} P_{\theta}\{Y_1 = y_1, \dots, Y_N = y_N | x_1, \dots, x_N\} \quad (2.11)$$

$$= \arg \max_{\theta} \prod_{n=1}^N P_{\theta}\{Y_n = y_n | x_n\} \quad (2.12)$$

$$= \arg \max_{\theta} \prod_{n=1}^N P_{\theta}\{Y_{n,1} = y_{n,1}, \dots, Y_{n,T} = y_{n,T} | x_{n,1}, \dots, x_{n,T}\} \quad (2.13)$$

$$= \arg \max_{\theta} \prod_{n=1}^N \prod_{t=1}^T P_{\theta}\{Y_{n,t} = y_{n,t} | x_{n,1}, \dots, x_{n,t}\} \quad (2.14)$$

Максимизирање веродостојности је еквивалентно минимизирању негативног логартма веродостојности јер је логаритам строго растућа функција.

$$\theta^* = \arg \max_{\theta} \prod_{n=1}^N \prod_{t=1}^T P_{\theta}\{\hat{Y}_{n,t} = y_{n,t} | x_{n,1}, \dots, x_{n,t}\} \quad (2.15)$$

$$= \arg \min_{\theta} - \ln \left(\prod_{n=1}^N \prod_{t=1}^T P_{\theta}\{\hat{Y}_{n,t} = y_{n,t} | x_{n,1}, \dots, x_{n,t}\} \right) \quad (2.16)$$

$$= \arg \min_{\theta} - \sum_{n=1}^N \sum_{t=1}^T \ln P_{\theta}\{\hat{Y}_{n,t} = y_{n,t} | x_{n,1}, \dots, x_{n,t}\} \quad (2.17)$$

$$(2.18)$$

Како су лабеле $y_{n,t}$ "one-hot" кодирани вектори, а излази из модела RNN $\hat{p}_{n,t}$ су предвиђена расподела вероватноће случајног вектора $Y_{n,t}$, дефинисаног једначином 2.8, можемо компактно написати:

$$P_{\theta}\{Y_{n,t} = y_{n,t} | x_{n,1}, \dots, x_{n,t}\} = \prod_{k=1}^K (\hat{p}_{n,t,k})^{y_{n,t,k}}$$

Сада проблем оптимизације постаје:

$$\begin{aligned} \theta^* &= \arg \min_{\theta} - \sum_{n=1}^N \sum_{t=1}^T \ln P_{\theta}\{Y_{n,t} = y_{n,t} | x_{n,1}, \dots, x_{n,t}\} \\ &= \arg \min_{\theta} \sum_{n=1}^N \sum_{t=1}^T - \sum_{k=1}^K y_{n,t,k} \ln \hat{p}_{n,t,k} \end{aligned}$$

Крос ентропија између вектора $y_{n,t}$ и $\hat{p}_{n,t}$ је дефинисана на следећи начин:

$$L(y_{n,t}, \hat{p}_{n,t}) = - \sum_{k=1}^K y_{n,t,k} \ln \hat{p}_{n,t,k} \quad (2.19)$$

Проблем оптимизације максимизирања веродостојности скупа података у односу на параметре θ је еквивалентан је проблему минимизирања збира функције цене унакрсне ентропије $L(y_{n,t}, \hat{p}_{n,t})$ између лабела $y_{n,t}$ и излаза RNN модела $\hat{p}_{n,t}$ за све узорке n и све временске кораке t .

$$\begin{aligned}\theta^* &= \arg \max_{\theta} P_{\theta}\{\hat{Y} = y|x\} \\ &= \arg \min_{\theta} \sum_{n=1}^N \sum_{t=1}^T L(y_{n,t}, \hat{p}_{n,t})\end{aligned}\tag{2.20}$$

Циљ фазе тренирања је проналажење оптималних параметара θ^* који минимизирају функцију цене дефинисану једначином 2.21. У пракси се за то користи градијентни спуст.

Комплексност времена извршавања пролаза сигнала унапред и грешке уназад је $O(T)$ и не може смањити паралелизацијом јер је граф израчунавања по дизајну секвенцијалан. Свака променљива у временском кораку t се може израчунати тек након израчунавања променљивих у претходним корацима. Скривена стања и излази израчунати у пролазу сигнала унапред се морају чувати док се поново не употребе током пролаза грешке уназад, тако да је трошак меморије такође $O(T)$. Алгоритам пропагације грешке уназад примењен на размотани граф, са комплексношћу $O(T)$, назива се **пропагација грешке уназад кроз време** (енгл. **Backpropagation through time**). Можемо приметити да је рекурентна неуронска мрежа веома моћна, али и скупа за израчунавање.

$$L(\theta) = \sum_{n=1}^N \sum_{t=1}^T L(y_{n,t}, \hat{p}_{n,t})\tag{2.21}$$

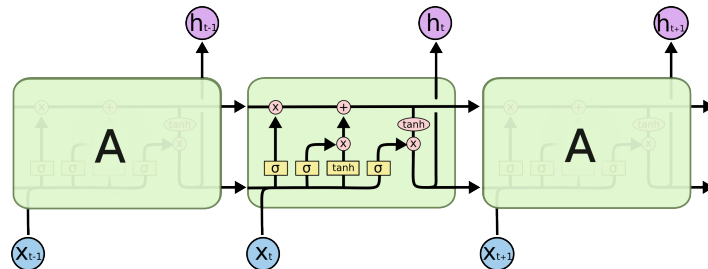
Проблем нестајања градијента

Проблем нестајања градијента током методе простирања грешке уназад кроз време (*BPTT*) је назив за феномен у коме градијенти параметара у временским корацима $t \gg 1$ веома слабо утичу на градијенте параметара првих временских корака. Ово је проблематично када моделирамо дугачак низ улаза јер ће ажурирања бити изузетно спора.

2.4.3 The Long Short-Term Memory

Управо због проблема нестајања градијента (Y. Bengio, Simard и Frasconi 1994.), ванила рекурентне неуронске мреже се ретко користе у пракси. Уместо тога, користи се *Long Short-Term Memory* (*LSTM*) мрежа, која је дизајнирана да реши поменуте проблеме (Sak, Senior и Beaufays 2014.a). *LSTM* мреже представљају посебну врсту *RNN*-а које могу да уче дугорочне зависности. *LSTM* мреже могу да моделују и до 1000 дискретних временских корака између релевантних улазних сигнала и излаза Hochreiter и Schmidhuber 1997., док су обичне *RNN* мреже ограничене на 5–10 временских корака (Sak, Senior и Beaufays 2014.b).

LSTM архитектура је изузетно популарна и користи се у разним проблемима моделирања секвенци, укључујући пуно проблема обраде природног језика у *Google-y* (Goodfellow, Yoshua Bengio и Courville 2016.).



Слика 2.23: LSTM блок. Source: Olah 2015.

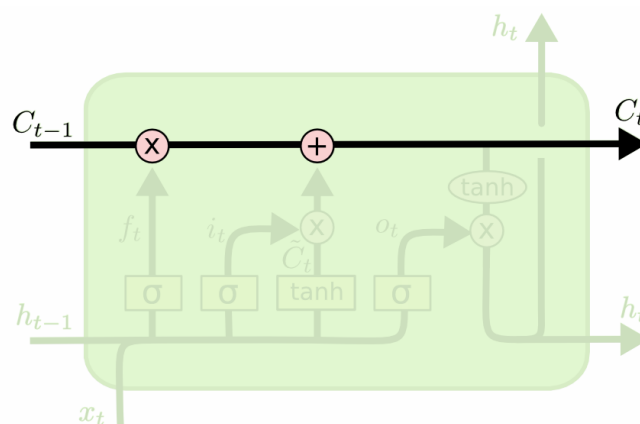
Ова секција објашњава компоненте LSTM-а - стање ћелије, улазну капију (енгл. input gate), капију заборављања (енгл. forget gate) и излазну капију (енгл. output gate). За сваку од компоненти дате су одговарајуће математичке формуле и објашњен процес тренирања.

Елементи LSTM-а

Нека је улазна секвенца $x = [x_1, x_2, \dots, x_T]$ састављена од T вектора димензије d_i . У проблему класификације фрејмова видео записа, улаз у временском кораку t је вектор x_t који представља улазну слику. Ако слика има димензије $(244, 244, 3)$, онда је $d_i = 244 \cdot 244 \cdot 3$.

Главни део LSTM блока је **стање ћелије** C_t . Меморијска ћелија кодира знање у сваком временском кораку о томе који су улази до тог тренутка већ били обрађени (Vinyals и др. 2015.). Можемо је замислити као покретну траку која пролази кроз цео ланац, са само мањим линеарним интеракцијама. Веома је лако да информација тече кроз њу без измена, као што је приказано на 2.24.

У задатку класификације фрејмова видео записа, стање ћелије чува информације о претходним кадровима који су повезани са тренутним.



Слика 2.24: Стање ћелије C_t у LSTM блоку. Извор: Olah 2015.

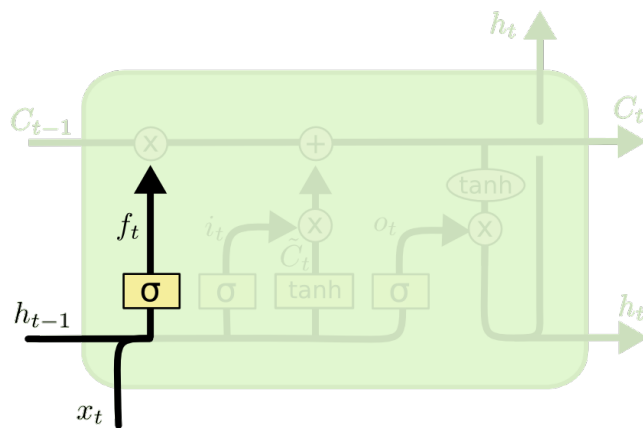
LSTM може да уклања или додаје информације у стање ћелије C_t на основу текућег улаза x_{t-1} и претходног излаза h_t . Уклањање информација регулише **капија заборављања** f_t , а додавање информација регулише **улазна капија** i_t , дефинисана у 2.22 и 2.23. Капије представљају механизам који омогућава контролисано пропуштање информација.

У једначинама 2.22 и 2.23, као и у наредним једначинама, $[h_{t-1}, x_t]$ означава конкатенацију (спајање) претходног скривеног стања и текућег улаза, док $\odot$ означава елемент-по-елемент множење (енгл. element-wise multiplication).

Одлука о томе које ће се информације из претходног стања ћелије C_{t-1} уклонити доноси се аутоматски путем капије заборављања f_t , што је илустровано у 2.25. Капија заборављања, дефинисана у 2.22, представља једноставан линеарни слој који као улазе прима претходни излаз h_t и текући улаз x_t , а као активациону функцију користи сигмоидну. Сигмоидна активациона функција обезбеђује да свака вредност у вектору f_t буде број између 0 (потпуно заборавити ову информацију из C_{t-1}) и 1 (потпуно запамтити ову информацију из C_{t-1}).

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (2.22)$$

У проблему класификације фрејмова видео записа, ако $f_t \approx 0$, то значи да је LSTM одлучио да претходни фрејмови $x_{t-1}, x_{t-2} \dots$ нису важни за тренутни x_t (на пример, претходни фрејмови су приказивали природу, а тренутни фрејм приказује ентеријер куће). С друге стране, ако $f_t \approx 1$, ово значи да је LSTM проценио да је информација из претходни фрејмова важна за обраду тренутног (на пример, фрејмови припадају истој сцени).



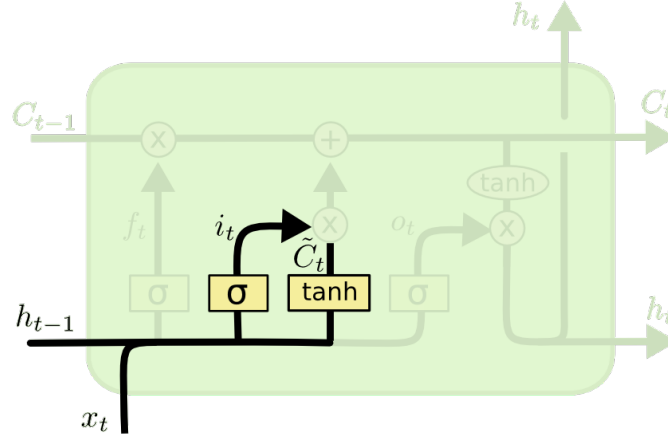
Слика 2.25: Капија заборављања $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$ у LSTM блоку. Source: Olah 2015.

Одлука о томе које ће информације бити додате (смештене) у текуће стање ћелије C_t на основу претходног излаза h_{t-1} и текућег улаза x_t доноси се аутоматски помоћу **улазне капије** i_t и вектора **нових кандидатских вредности** (енгл. candidate values) $\tilde{C}_t$, као што је приказано на 2.27.

Улазна капија, дефинисана у 2.23 и илустрована у 2.26, исто као и капија заборављања, представља једноставан линеарни слој који као улазе прима претходни излаз h_t и текући улаз x_t , а као активациону функцију користи сигмоидну.

Сигмоидна активациона функција обезбеђује да свака вредност у вектору i_t буде број између 0 (не додај ову информацију из $\tilde{C}_t$) и 1 (потпуно додај ову информацију из $\tilde{C}_t$).

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (2.23)$$



Слика 2.26: Улазна капија $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$ и нови кандидат $\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_i)$ у LSTM блоку. Source: Olah 2015.

У задатку класификације фрејмова видео записа, ако улазна капија врати $i_{t,k} \approx 0$ за неку улазну карактеристику $x_{t,k}$, то значи да је LSTM одлучио да та специфична карактеристика није довољно значајна да би се сачувала у стању ћелије C_t .

Линеарни слој са $\tanh$ као активационом функцијом производи вектор нових кандидатских вредности $\tilde{C}_t$ користећи претходни излаз h_t и текући улаз x_t као улазе, као што је дефинисано у 2.24 и илустровано у 2.26.

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_i) \quad (2.24)$$

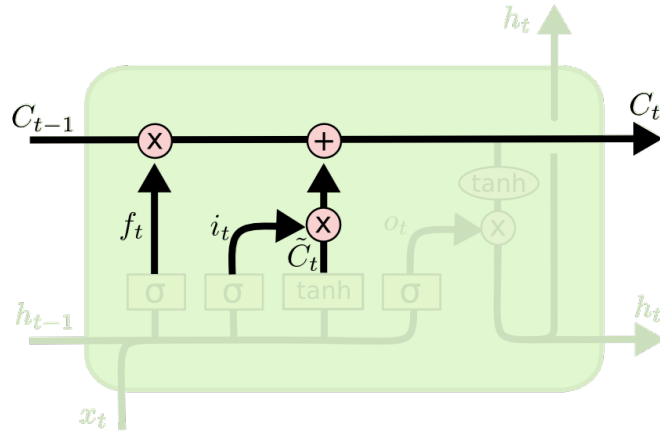
Да би се старо стање ћелије C_{t-1} ажурирало у ново стање C_t , примењује се 2.25, где $\odot$ представља множење по елементима. Процес ажурирања је приказан у ?? 2.27.

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (2.25)$$

На крају, одлука о томе шта ће бити излаз из LSTM блока доноси се помоћу **излазне капије** o_t и згњеченог стања ћелије C_t .

Излазна капија дефинисана у 2.26 и илустрована у 2.28.

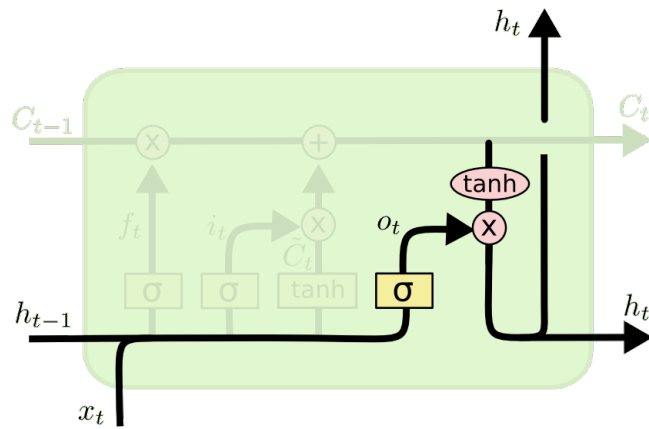
Вредности у вектору излазних врата o_t су бројеви између 0 и 1. Вредност 0 значи да информације из текућег стања ћелије C_t уопште нису коришћене за



Слика 2.27: Ажужирати стања ћелије $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$ у LSTM блоку. Source: Olah 2015.

тренутни излаз h_t , док вредност 1 у o_t значи да је целокупна информација из C_t искоришћена за h_t .

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (2.26)$$



Слика 2.28: Излазна капија $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$ у LSTM блоку. Source: Olah 2015.

Излаз LSTM блока је скривено стање h_t , дефинисано у 2.27. Скривено стање h_t се такође користи у наредном временском кораку $t+1$, као што је илустровано у 2.28 и 2.23.

$$h_t = o_t \odot \tanh(C_t) \quad (2.27)$$

Проблем класификације фрејмова видео записа представља мултикласну класификацију, па излаз o_t мора бити прослеђен кроз Softmax активациону функцију, чиме се добија расподела излаза преко свих могућих класа:

$$\hat{p}_t = \text{softmax}(h_t) \quad (2.28)$$

Једначине 2.22, 2.23, 2.26, 2.24, 2.27, и 2.28 дефинишу **forward pass** за мултикласну класификацију кроз LSTM мрежу, заједно наведено у 3.1.

$$\begin{aligned} f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \\ i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \\ \tilde{C}_t &= \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \\ C_t &= f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \\ o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \\ h_t &= o_t \odot \tanh(C_t) \\ \hat{p}_t &= \text{softmax}(h_t) \end{aligned} \quad (2.29)$$

У 3.1 $[h_{t-1}, x_t]$ означава конкатенацију претходног скривеног стања и текућег улаза, а $\odot$ означава множење по елементима.

Скривено стање h_t у временском кораку t је d_h -димензионални вектор, где је d_h хиперпараметар. Сви посредни резултати: капије f_t, i_t, o_t , кандидатске вредности $\tilde{C}_t$ и стање ћелије C_t у временском кораку t такође су d_h -димензионални вектори.

Матрице тежина W_f, W_i, W_o, W_c имају димензије $d_h \times (d_h + d_i)$, а вектори помака b_f, b_i, b_o, b_c су d_h -димензионални.

Тренирање LSTM

Параметри који се могу научити у LSTM блоку су матрице тежина и вектори улазних, излазних и капија заборављања $W_f, b_f, W_i, b_i, W_o, b_o$, као и матрица тежина и вектор пристрасности кандидата меморије W_c, b_c . За компакт-нију нотацију, спајамо све параметре у један тензор који се учи током процеса тренирања:

$$\theta = [W_f, b_f, W_i, b_i, W_o, b_o, W_c, b_c]$$

Изразе из модела $\hat{p}_t$ тумачимо као предвиђања модела расподеле случајне промењиве $\hat{Y}_t$ за све могуће класе, дефинисане као ??

$$P(Y_{t,k} = 1) = \begin{cases} \hat{p}_{t,1}, & \text{if } k = 1 \\ \hat{p}_{t,2}, & \text{if } k = 2 \\ \dots & \\ \hat{p}_{t,K}, & \text{if } k = K \end{cases}$$

Циљ тренирања LSTM мреже је проналажење параметара θ^* који максимизирају веродостојност скупа података за тренирање величине N , дефинисаног са:

$$D = \{(x_1^n, \dots, x_T^n), (y_1^n, \dots, y_T^n)\}_{n=1}^N$$

$$\theta^* = \arg \max_{\theta} P_{\theta}\{Y = y|x\} \quad (2.30)$$

$$= \arg \max_{\theta} P_{\theta}\{Y_1 = y_1, \dots, Y_N = y_N|x_1, \dots, x_N\} \quad (2.31)$$

Слично корацима из тренирања ванила рекурентних неуронских мрежа, долазимо до еквивалентног оптимизационог проблема:

$$\theta^* = \arg \max_{\theta} P_{\theta}\{Y = y|x\} \quad (2.32)$$

$$= \arg \min_{\theta} \sum_{n=1}^N \sum_{t=1}^T L(y_{n,t}, \hat{p}_{n,t}) \quad (2.33)$$

у којем је функција цене сума крос ентропија између вектора $y_{n,t}$ и $\hat{p}_{n,t}$:

$$L(y_{n,t}, \hat{p}_{n,t}) = - \sum_{k=1}^K y_{n,t,k} \ln \hat{p}_{n,t,k} \quad (2.34)$$

Оптимизациони проблем максимизирања веродостојности датог скупа података по параметрима θ , дат једначином 2.30, је еквивалентан проблему минимизирања збира цене грешке унакрсне ентропије $L(y_{n,t}, \hat{p}_{n,t})$ између лабела $y_{n,t}$ и LSTM-ових предвиђања $\hat{p}_{n,t}$ за све могуће узорке n и за све временске кораке t , дат једначином 2.34.

Циљ током фазе тренирања је проналажење оптималних параметара θ^* који минимизирају функцију цене, дефинисану једначином 2.35, користећи пропагацију грешке уназад.

$$L(\theta) = \sum_{n=1}^N \sum_{t=1}^T L(y_{n,t}, \hat{p}_{n,t}) \quad (2.35)$$

Оптимизација за дугорочне зависности

С обзиром на дефиницију стања ћелије, дату једначином 2.25, информације лакше теку од једног временског корака t до следећег $t+1$. На овај начин, LSTM има непрекинути градијентни ток где се градијенти пропагирају уназад кроз стања ћелија уместо кроз скривена стања h_t , без нестајања у сваком временском кораку.

Оваква архитектура у великој мери решава проблем нестајања и експлозије градијента, описан у подсекцији 2.4.2. Теоретски, LSTM архитектура не гарантује да неће бити проблема нестајања/експлозије градијента, али пружа лакши начин да модел научи зависности код секвенци великих дужина (Johnson 2021.).

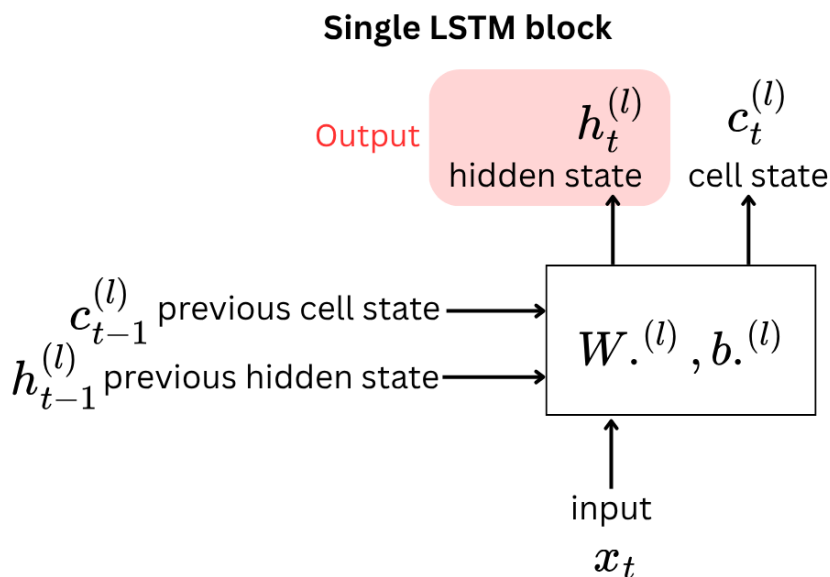
Вишеслојни LSTM

Вишеслојна LSTM мрежа, такође позната као наслагани LSTM (енгл. stacked LSTM), састоји се од више LSTM блокова (слојева) наслаганих један на други. Један LSTM блок је приказан на слици 2.29, а вишеслојна LSTM мрежа је приказана на слици 2.30. Ова архитектура омогућава мрежи да научи сложене временске правилности креирањем хијерархијских репрезентација улазних података. Развијена вишеслојна LSTM мрежа је приказана на слици 2.31.

У вишеслојној LSTM мрежи, излаз из једног LSTM слоја служи као улаз за следећи LSTM слој у истом временском кораку. Означимо слојеве горњим индексом l , где $l = 1$ означава први слој, а $l = L$ последњи слој.

Нотација за вишеслојни LSTM:

- $x_t^{(l)}$ - Улазни вектор у слој l у временском кораку t
- $h_t^{(l)}$ - Скривени вектор стања слоја l у временском кораку t
- $c_t^{(l)}$ - Вектор стања ћелије слоја l у временском кораку t



Слика 2.29: Вишеслојни LSTM блок

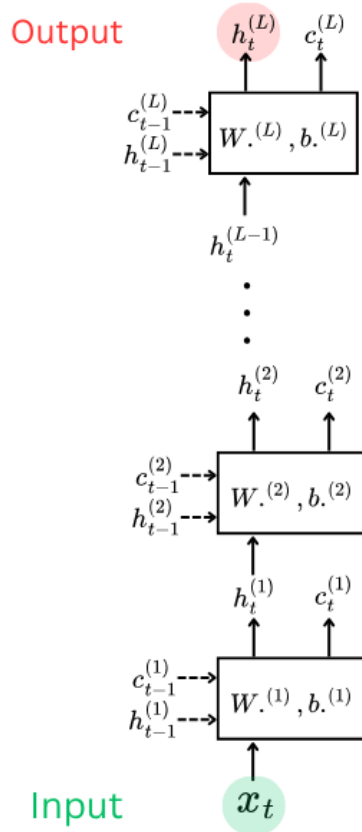
Forward pass једног LSTM блока у l -том слоју и временском кораку t је дефинисан једначинама 2.36.

$$\begin{aligned}
f_t^{(l)} &= \sigma(W_f^{(l)} \cdot [h_{t-1}^{(l)}, x_t^{(l)}] + b_f^{(l)}) \\
i_t^{(l)} &= \sigma(W_i^{(l)} \cdot [h_{t-1}^{(l)}, x_t^{(l)}] + b_i^{(l)}) \\
\tilde{C}_t^{(l)} &= \tanh(W_c^{(l)} \cdot [h_{t-1}^{(l)}, x_t^{(l)}] + b_c^{(l)}) \\
C_t^{(l)} &= f_t^{(l)} \odot C_{t-1}^{(l)} + i_t^{(l)} \odot \tilde{C}_t^{(l)} \\
o_t^{(l)} &= \sigma(W_o^{(l)} \cdot [h_{t-1}^{(l)}, x_t^{(l)}] + b_o^{(l)}) \\
h_t^{(l)} &= o_t^{(l)} \odot \tanh(C_t^{(l)})
\end{aligned} \tag{2.36}$$

Број слојева L је хиперпараметар.

Величина улазних вектора у првом $l = 1$ LSTM блоку, означених са $x_t, \forall t = \overline{1, T}$, је d_i . Величина скривених стања, означених са $h_t^{(l)}, \forall t = \overline{1, T}, \forall l = \overline{1, L}$, је хиперпараметар d_h . Величина улазних вектора у другом, трећем, ..., L -том LSTM блоку је $h_t^{(l)}, \forall t = \overline{1, T}, \forall l = \overline{2, L}$ је d_h .

Multilayer LSTM

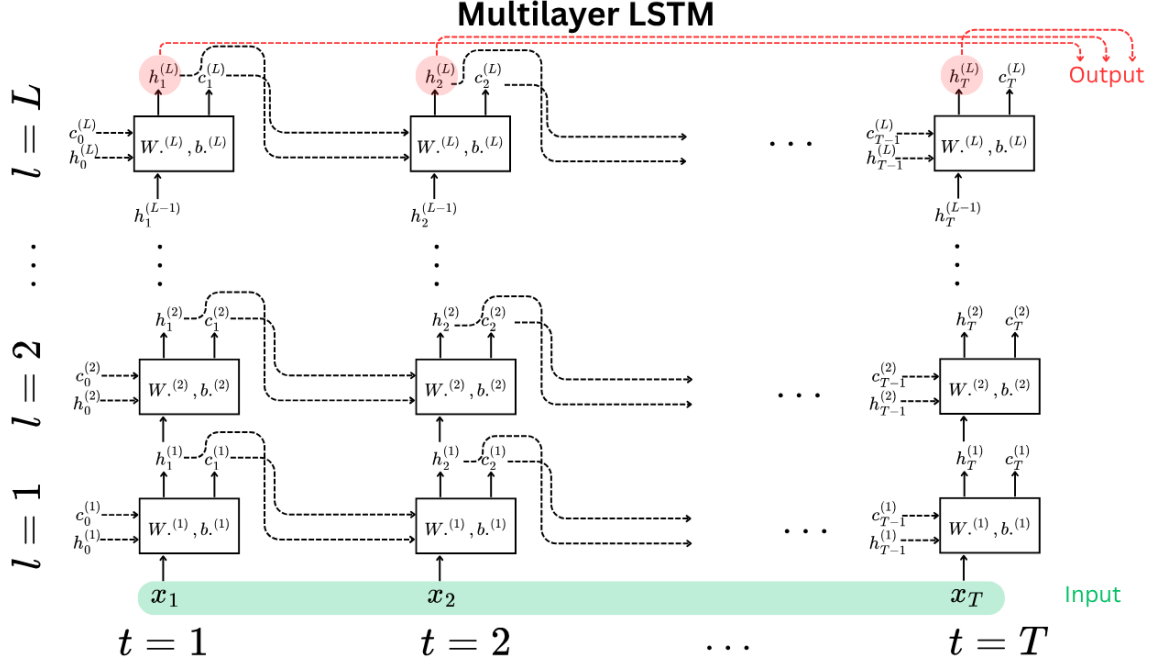


Слика 2.30: Вишеслојни LSTM

Ток информација у вишеслојној LSTM структури, приказан на слици 2.30, прати следеће кораке:

- **У временском кораку $t = 1$, за први слој $l = 1$:** Улази су прва вредност улазне секвенце x_1 , иницијализовано скривено стање $h_0^{(1)}$ и стање ћелије $c_0^{(1)}$. Најчешће подразумеване вредности су нулти вектори: $h_0^{(1)} = 0$ и $c_0^{(1)} = 0$. Прорачуни LSTM ћелија (капије заборављања, улазне капије, стање ћелија кандидата, ажурирање стања ћелија, излазне капије, скривена стања) се изводе као што је описано у једној LSTM ћелији ?? 3.1, користећи надскрипт (1) за сва стања, тежине и помераје. Излази овог LSTM блока су скривено стање $h_1^{(1)}$ и стање ћелије $c_1^{(1)}$.
- **У временском кораку $t = 1$, за други слој $l = 2$:** Улази су скривено стање $h_1^{(1)}$ из претходног блока, иницијализовано скривено стање $h_0^{(2)}$ и стање ћелије $c_0^{(2)}$. Најчешће подразумеване вредности су нулти вектори: $h_0^{(2)} = 0$ и $c_0^{(2)} = 0$. Прорачуни LSTM ћелија се изводе као што је описано у једној LSTM ћелији ?? 3.1, користећи надскрипт (2) за сва стања, тежине и помераје. Излази овог LSTM блока су скривено стање $h_1^{(2)}$ и стање ћелије $c_1^{(2)}$.
- ...
- **У временском кораку $t = 1$, за последњи слој $l = L$:** Улази су скривено стање $h_1^{(L-1)}$ из претходног блока, иницијализовано скривено стање $h_0^{(L)}$ и стање ћелије $c_0^{(L)}$. Прорачуни LSTM ћелија се изводе као што је описано у једној LSTM ћелији ?? 3.1, користећи надскрипт (L) за сва стања, тежине и помераје. Излази овог LSTM блока су скривено стање $h_1^{(L)}$ и стање ћелије $c_1^{(L)}$.
- ...
- **У временском кораку $t = 2$, за први слој $l = 1$:** Улази су друга вредност улазне секвенце x_2 , као и скривено стање $h_1^{(1)}$ и стање ћелије $c_1^{(1)}$ из претходног временског корака. LSTM прорачуни се врше коришћењем надскрипта (1) за сва стања, тежине и помераје. Излази овог LSTM блока су скривено стање $h_2^{(1)}$ и стање ћелије $c_2^{(1)}$.
- **У временском кораку $t = 2$, за други слој $l = 2$:** Улази су скривено стање $h_2^{(1)}$ из претходног блока, као и скривено стање $h_1^{(2)}$ и стање ћелије $c_1^{(2)}$ из претходног временског корака. LSTM прорачуни се врше коришћењем надскрипта (2) за сва стања, тежине и помераје. Излази овог LSTM блока су скривено стање $h_2^{(2)}$ и стање ћелије $c_2^{(2)}$.
- ...
- ...

- У временском кораку $t = 2$, за последњи слој $l = L$: Улази су скривено стање $h_2^{(L-1)}$ из претходног блока, као и скривено стање $h_1^{(L)}$ и стање ћелије $c_1^{(L)}$ из претходног временског корака. LSTM прорачуни ћелија се врше коришћењем надскрипта (L) за сва стања, тежине и помераје. Излази овог LSTM блока су скривено стање $h_2^{(L)}$ и стање ћелије $c_2^{(L)}$.



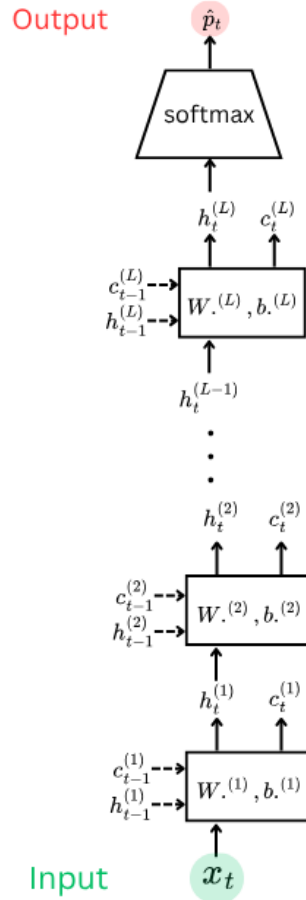
Слика 2.31: Развијени вишеслојни LSTM

Архитектура вишеслојне LSTM мреже је конструисана тако да је улазна величина (величина улазног вектора) слоја l (за $l > 1$) експлицитно подешена да буде једнака скривеној величини претходног слоја $l - 1$.

За проблем вишекласне класификације, као што је задатак класификације видео фрејмова, на врх L -тог LSTM блока у сваком временском кораку се додаје линеарни слој са Softmax активационом функцијом, као што је приказано на слици 2.32. Излаз у сваком временском кораку t је расподела по свим могућим класама $\hat{p}_t$, дефинисана једначином 2.37. Матрица тежина W је димензија $K \times d_h$, а вектор помераја b је K -димензионални вектор, где је K број класа.

$$\hat{p}_t = \text{softmax}(Wh_t^{(L)} + b) \quad (2.37)$$

Multilayer LSTM for multiclass classification



Слика 2.32: Вишеслојна LSTM мрежа за задатак вишекласне класификације, као што је класификација видео фрејмова. Излаз у сваком временском кораку t је расподела по свим могућим класама $\hat{p}_t = \text{softmax}(Wh_t^{(L)} + b)$.

2.5 BLEU метрика

BLEU (*Bilingual Evaluation Understudy*) је **метрика** за процену квалитета аутоматски преведеног текста, која пореди машински генерисан текст са једним или више референтних, људски преведених текстова. Поред аутоматског преводјења, користи се и у другим задацима обраде природног језика (*NLP*), као што је генерисање описа слике. Резултат се креће од 0 до 1, где виши резултат указује на већу сличност са референтним текстовима, а израчунава се поређењем n -грамова (секвенци речи) и применом казне за прекратке преводе. Коришћење *BLEU* метрике за евалуацију је аутоматско, брзо и ефикасно, али не узима у обзир семантичко значење речи, ни граматику текста. Уведимо *BLUE* метрику кроз пример.

- Аутоматски генерисан опис: "Жена са кишобраном стоји на улици."

- Референтни текст (анотација): "Жена са розе кишобраном стоји на улици"

n -грам је секвенца од n речи, постоје униграми ($n = 1$), биграми ($n = 2$), 3-грами ($n = 3$), 4-грами ($n = 4$)... За аутоматски генерисан опис "Жена са кишобраном стоји на улици." n -грами су:

- **униграми:** "Жена", "са", "кишобраном", "стоји", "на", "улици"
- **биграми:** "Жена са", "са кишобраном", "кишобраном стоји", "стоји на", "на улици"
- **3-грами:** "Жена са кишобраном", "са кишобраном стоји", "кишобраном стоји на", "стоји на улици"
- **4-грами:** "Жена са кишобраном стоји", "са кишобраном стоји на", "кишобраном стоји на улици"
- **5-грами:** "Жена са кишобраном стоји на", "са кишобраном стоји на улици"
- **6-грами:** "Жена са кишобраном стоји на улици"

Скуп n -грама аутоматски генерисаног текста (*candidate*) означимо са C_n , а скуп n -грама референтни текст (*reference*) R_n . За пример изнад важи:

- $C_1 = \{ \text{"Жена", "са", "кишобраном", "стоји", "на", "улици"} \}$
- $R_2 = \{ \text{"Жена", "са", "розе", "кишобраном", "стоји", "на", "улици"} \}$

Дефинишимо n -грам прецизност (енгл. n -gram precision score) између аутоматски генерисаног текста и референтног текста:

$$P_n = \frac{|\{x \in C_n | x \in R_n\}|}{|C_n|}$$

За пример изнад важи:

- $P_1 = \frac{6}{6} = 1$
- $P_2 = \frac{4}{5} = 0.8$
- $P_3 = \frac{2}{4} = 0.5$
- $P_4 = \frac{1}{3} \approx 0.33$
- $P_5 = \frac{0}{2} \approx 0$
- $P_6 = \frac{0}{1} \approx 0$

Прецизност P се дефинише као геометријска средина ненула n -грам прецизности:

$$P = (P_1 \cdot P_2 \cdot P_3 \cdot P_4)^{\frac{1}{4}} \approx (1 \cdot 0.8 \cdot 0.5 \cdot 0.33)^{\frac{1}{4}} \approx 0.60$$

Ако је аутоматски генерисан опис: "Жена на улици.", n -грам прецизности би биле:

- $P_1 = \frac{3}{3} = 1$
- $P_2 = \frac{2}{2} = 1$
- $P_3 = \frac{1}{1} = 1$

Средња прецизност би била:

$$P = (P_1 \cdot P_2 \cdot P_3)^{\frac{1}{3}} = (1 \cdot 1 \cdot 1)^{\frac{1}{3}} = 1$$

Примећујемо да n -грам прецизност фафоризује кратке описе. Због тога се уводи **казна за кратке описе** (енгл. *Brevity Penalty*) B_p :

$$B_p := \begin{cases} 1, & \text{ако } c > r \\ e^{\left(1 - \frac{r}{c}\right)}, & \text{ако } c \leq r \end{cases}$$

где је c дужина аутоматски генерисаног описа, а r дужина референтног описа.

BLEU метрика се дефинише на следећи начин:

$$\text{BLEU} = B_p \cdot P$$

За пример:

- Аутоматски генерисан опис: "Жена на улици."
- Референтни текст (анотација): "Жена са розе кишобраном стоји на улици",

казна за кратак опис и **BLEU** метрика се рачунају на следећи начин:

$$\begin{aligned} c &= 3 \\ r &= 7 \\ B_p &= e^{\left(1 - \frac{7}{3}\right)} = e^{-\frac{4}{3}} \approx 0.26 \end{aligned}$$

$$\text{BLEU} = B_p \cdot P \approx 0.26 \cdot 1 = 0.26$$

За пример:

- Аутоматски генерисан опис: "Жена са кишобраном стоји на улици."
- Референтни текст (анотација): "Жена са розе кишобраном стоји на улици"

казна за кратак опис је јенака:

$$\begin{aligned} c &= 6 \\ r &= 7 \\ B_p &= e^{\left(1 - \frac{7}{6}\right)} = e^{-\frac{1}{6}} \approx 0.85 \end{aligned}$$

$$\text{BLEU} = B_p \cdot P \approx 0.85 \cdot 0.6 \approx 0.51$$

У скупу података за тренирање се обично налази **више референтних опис** за једну слику. Наведимо пример:

- Аутоматски генерисан опис: "Жена на улици."
- Први референтни текст (анотација): "Жена са розе кишобраном стоји на улици"
- Други референтни текст (анотација): "Жена у кишном мантилу са кишобраном"

У том случају за P_n се узима максимална P_n међу датим, а при рачунању казне за кратак опис за r се узима вредност најближа c . За пример:

- Аутоматски генерисан опис: "Жена са кишобраном стоји на улици."
- Референтни текст (анотација): "Жена са розе кишобраном стоји на улици"
- Други референтни текст (анотација): "Жена у кишном мантилу са кишобраном"

важи:

$$\begin{aligned}
 P_1 &= \max\{1, 0.5\} = 1 \\
 P_2 &= \max\{0.8, 0.2\} = 0.8 \\
 P_3 &= \max\{0.5, 0\} = 0.5 \\
 P_4 &= \max\{0.33, 0\} = 0.33 \\
 P &= 0.6 \\
 c &= 6 \\
 r &= 6 \\
 B_p &= 1 \\
 \text{BLEU} &= B_p \cdot P \approx 1 \cdot 0.6 \approx 0.6
 \end{aligned}$$

Напоменимо да BLEU метрика не задовољава услове математичке дефиниције метрике, иако се тако зове.

Глава 3

Методологија

У овој глави објашњен је цео процес изградње система за аутоматско генерисање описа слике од почетка до краја. У поглављу 3.1 је описан скуп података за тренирање и алгоритми за препроцесуирање описа и слика, у поглављу 3.2 је описана архитектура модела и *forward pass*, у поглављу је описан оптимизациони проблем и алгоритам за његово решавање (тренирање модела) 3.3, у поглављу 3.4 је приказано како се истренирани модели користе за генерисање описа слике и у поглављу 3.5 је описан процес евалуације натренираног модела.

3.1 Скуп података

3.1.1 Препроцесуирање слика и описа

Натренирани (енгл. *pre-trained*) ResNet модел на улазу узима слику фиксних димензија (3, 224, 224). Такође, даје најбоље резултате када су слике нормализоване по каналима, тако да средња вредност буде [0.485, 0.456, 0.406], а стандардна девијација [0.229, 0.224, 0.225]. Висина и ширина слике у МС СОСО скупа података за тренирање варира, као што је објашњено у глави 2.1. Због тога дефинишемо *preprocessing pipeline* за слику:

1. Промена величине улазне слике тако да мања ивица буде 256 пиксела, уз очување односа ширине и висине.
2. Задржава се централни део слике фиксне величине 224×224 пиксела.
3. Скалирање вредности сваког пиксела од [0, 255] до [0.0, 1, 0].
4. Нормализација сваког од три канала тако да средња вредност буде [0.485, 0.456, 0.406], а стандардна девијација [0.229, 0.224, 0.225].

Након препроцесуирања слике су структурисане и можемо да их именујемо. Скуп слика за тренирање означавамо са:

$$\{x_n | n \in 1, 2, \dots, N\},$$

где је x_n једна слика димензија (3, 224, 224).

Да би модел "нахранли" реч по реч, свака реч мора да се индексира, тј. свакој речи треба да се додели целобројна вредност. Зато пре процесуирања описа креирамо вокабулар тако што направимо скуп свих речи које се јављају. Сортирамо их абecedно и свакој доделимо јединствену целобројну вредност. Након креирања вокабулара једнозначно су дефинисана пресликавања из скупа речи у скуп индекса и из скупа индекса у скуп речи.

Опис слике у скупу података за тренирање се састоји од речи и знакова интерпункције. Има речи које почињу малим и великим словом. Дужина описа варира од 5 до 49 речи, детаљније објашњено у 2.1. Због тога дефинишемо *preprocessing pipeline* за опис:

1. Брисање знакова интерпункције.
2. Сва слова постају мала.
3. Опис се токенизује. Сваки опис (string) се конвертује у листу речи (string-ова).
4. Речи се индексирају. Сваки опис постаје листа индекса.
5. Додају се специјални токени који означавају почетак (1) и крај (2). Сваки опис постаје листа индекса која почиње са 1 и завршава се са 2.
6. Описи се допуњују специјалним токенима (0) тако да је сваки опис једнаке дужине L .

Након препроцесуирања описи су структурисани и можемо да их именујемо. Скуп описа за тренирање означавамо са:

$$\{y_n | n \in 1, 2, \dots, N\},$$

где је y_n један опис, представљен као вектор дужине L .

Узорак у скупу података за тренирање (након препроцесуирања) је уређени пар слике и описа (x_n, y_n) . **Скуп података за тренирање** означавамо са $D = \{(x_n, y_n) | n \in 1, 2, \dots, N\}$.

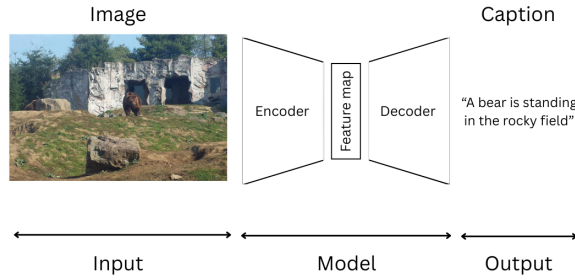
Обим MC COCO скупа за тренирање је $N = 118.287$, **обим скупа за валидацију** је $N_{val} = 2.500$ и **обим скупа за тестирање** је $N_{test} = 2.500$.

3.2 Архитектура модела

Архитектура модела за генерисање описа слике је енкодер-декодер, детаљно објашњен у поглављу 2.2.

3.2.1 Енкодер-декодер

Енкодер-декодер модел је "sequence-to-sequence" модел, на улазу узима секвенцу x_n и на излазу производи секвенцу y_n . Код генерисања описа слике улазна секвенца је препроцесуирана слика $x_n \in R^{3 \cdot 244 \cdot 224}$, а излазна секвенца је препроцесуира опис $y_n \in R^L$, као што је приказано на слици 3.1.



Слика 3.1: Енкодера-декодера модел за генерисање описа слика.

Енкодер процесуира улазну слику x_n и производи вектор карактеристика (енгл. "*feature vector*") z_n фиксне дужине M , што је приказано на слици 3.1. Вектор карактеристика z_n садржи есенцијалне информације о улазној слици x_n (шта се налази на слици, које активности су на слици, које боје су објекти...).

Декодер као улаз узима вектор карактеристика z_n , а производи опис, реч по реч.

3.2.2 Енкодер

Енкодер модела се заснива на натрениранираном *ResNet* моделу (He и др. 2016.) детаљно објашњеног у глави 2.3. Енкодер чине сви слојеви *ResNet* модела, сем последњег, линеарног, слоја, а уместо њега се додаје линеарни слој који има E неурона на излазу и *batch normalization* слоја. E је хиперпараметар који представља димензију ембедованог простора у који се сликају и слике (коришћењем енкодера) и речи (коришћењем ембедованог слоја).

Нека је x_n препроцесуирана слика димензија $3 \times 224 \times 224$. Енкодер слика x_n у вектор карактеристика $z_n \in R^E$.

У енкодеру се користе се натрениране тежине *ResNet* модела које се не тренирају додатно. Једини параметри у енкодеру који се тренирају су матрица тежине и пристрасност последњег линеарног слоја и параметри последњег *batch normalization* слоја.

3.2.3 Декодер

Свака реч y_t описа y се преслика најпре у *one-hot-encoded* вектор димензије вокабулара V , а затим у ембедовани простор димензије E . **Ембединг** може да буде једноставан линеаран слој који на улазу V , а на излазу E неурона, или нека комплекснија неуронска мрежа.

Ради једноставнијег записа од сада ћемо са $y_t \in R^E$ означавати емедовану реч на позицији t у опису y дужине T .

У основи декодера је **LSTM** модел детаљније објашњен у поглављу 2.4. Означимо са x_t^{LSTM} улаз у *LSTM* у тренутку t је:

$$x_t^{LSTM} := \begin{cases} z, & t = 0 \\ y_t, & t \in \{1, 2, \dots, T - 1\}, \end{cases}$$

где је z вектор карактеристика (тј. излаз из енкодера за улазну слику x), T дужина реченице y са додатим специјалним токенима за почетак и крај, y_t емедована реч на позицији t у опису y .

Forward pass LSTM-а у тренутку t је дефинисан на следећи начин:

$$\begin{aligned} f_t &= \sigma(W_f \cdot [h_{t-1}, x_t^{LSTM}] + b_f) \\ i_t &= \sigma(W_i \cdot [h_{t-1}, x_t^{LSTM}] + b_i) \\ \tilde{C}_t &= \tanh(W_c \cdot [h_{t-1}, x_t^{LSTM}] + b_c) \\ C_t &= f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \\ o_t &= \sigma(W_o \cdot [h_{t-1}, x_t^{LSTM}] + b_o) \\ h_t &= o_t \odot \tanh(C_t) \end{aligned} \tag{3.1}$$

Након тога, скривено стање h_t пролази кроз **линеаран слој** са V неурона на излазу и *softmax* активационом функцијом. Улога овог слоја је да класификује излазну реч. Означимо са $\hat{p}_t$ излаз из линеарног слоја за улаз $y_t, \forall t \in \{1, 2, \dots, T - 1\}$. Вектор $\hat{p}_t$ интерпретирамо као функцију расподеле, где је на i -тој позицији вектора $\hat{p}_t$ вероватноћа да је реч y_t баш реч означена индексом i у вокабулару.

Декодер који је на улазу примио вектор карактеристика z производи секвенцу вектора $\hat{p} = [\hat{p}_1, \hat{p}_2, \dots, \hat{p}_{T-1}]$ као излаз.

Forward Pass

Узорак у скупу података за тренирање (након препроцесуирања) је $(x_n, y_n), n \in \{1, 2, \dots, N\}$, где је x_n препроцесуирана слика фиксне димензије, а $y_n = [y_{n,1}, y_{n,2}, \dots, y_{n,T}]$ је лабела дата као секвенца ембедованих речи са специјалним токенима који означавају почетак и крај.

У енкодер се проследи слика x_n као улаз и добија се вектор карактеристика z_n као излаз. Секвенца $[z_n, y_{n,1}, y_{n,2}, \dots, y_{n,T-1}]$ се проследи декодеру као улаз и он производи секвенцу $\hat{p} = [\hat{p}_1, \hat{p}_2, \dots, \hat{p}_{T-1}]$ као излаз.

3.3 Тренирање

3.3.1 Параметри модела за тренирање

Параметри модела за тренирање су параметри линеарног слоја и слоја нормализације у енкодеру и параметри ембединга, *LSTM*-а и класификационог линеарног слоја. Означимо са θ конкатениране у вектор параметре модела који се тренирају.

Параметри модела за тренирање су параметри линеарног слоја и слоја нормализације у енкодеру и параметри ембединга, *LSTM*-а и класификационог линеарног слоја. Означимо са θ конкатениране у вектор параметре модела који се тренирају.

Forward Pass Узорак у скупу подататака за тренирање (након препроцесуирања) је (x_n, y_n) , $n \in \{1, 2, \dots, N\}$, где је x_n препроцесуирана слика фиксне димензије, а $y_n = [y_{n,1}, y_{n,2}, \dots, y_{n,T}]$ је лабела дата као секвенца ембедованих речи са специјалним токенима који означавају почетак и крај.

У енкодер се проследи слика x_n као улаз и добија се вектор карактеристика z_n као излаз. Секвенца $[z_n, y_{n,1}, y_{n,2}, \dots, y_{n,T-1}]$ се проследи декодеру као улаз и он производи секвенцу $\hat{p} = [\hat{p}_1, \hat{p}_2, \dots, \hat{p}_{T-1}]$ као излаз.

3.3.2 Пробабилистичка интерпретација модела

Енкодер-декодер модел је пробабилистички модел, он производи функцију расподеле $\hat{p}_{n,t}$ случајног вектора $Y_{n,t}$ који представља реч на месту t , $t \in \{1, 2, \dots, T\}$ у опису слике x_n .

Излаз модела $p_{n,t,v}$ интерпретирамо као вероватноћу да је у узорку n у опису на месту t реч која је у вокабулару означена са v , тј:

$$\hat{p}_{n,l,v} = P_{\theta}\{Y_{n,l,v} = 1|x_n\},$$

где $\hat{p}_{n,l,v}$ очигледно зависи од параметара модела θ .

Претпоставка модела је да су узорци независни.

3.3.3 Метод максималне веродостојности

Да бисмо пронашли параметре модела θ , користимо метод максималне веродостојности (енгл. *Maximum Likelihood Estimation*):

$$\begin{aligned}\theta^* &= \arg \max_{\theta} L(\theta; D) \\ &= P_{\theta}\{Y = y|x\} \\ &= \arg \max_{\theta} P_{\theta}\{Y_1 = y_1, \dots, Y_N = y_N|x_1, \dots, x_N\} \\ &= \arg \max_{\theta} \prod_{n=1}^N P_{\theta}\{Y_n = y_n|x_n\}\end{aligned}$$

Минимизирање негативног логаритма веродостојности је математички еквивалентно максимизирању саме функције веродостојности јер је логаритам мононо растућа функција.

$$\begin{aligned}\theta^* &= \arg \min_{\theta} -\ln L(\theta; D) \\ &= \arg \min_{\theta} -\ln \prod_{n=1}^N P_{\theta}\{Y_n = y_n|x_n\} \\ &= \arg \min_{\theta} -\sum_{n=1}^N \ln P_{\theta}\{Y_n = y_n|x_n\} \\ &= \arg \min_{\theta} -\sum_{n=1}^N \ln P_{\theta}\{Y_{n,1} = y_{n,1}, \dots, Y_{n,T} = y_{n,T}|x_n\}\end{aligned}$$

С обзиром на то да је декодер чине рекуренте неуронске мреже (RNN), предикција модела за реч у кораку t зависи од свих претходних скривених стања $1, 2, \dots, t-1$. Користећи Бајесову теорему, имамо:

$$\begin{aligned}
\theta^* &= \arg \min_{\theta} -\ln L(\theta; D) \\
&= \arg \min_{\theta} -\sum_{n=1}^N \ln P_{\theta}\{\hat{Y}_{n,1} = y_{n,1}, \dots, \hat{Y}_{n,L_2} = y_{n,L_2} | x_n\} \\
&= -\arg \min_{\theta} \sum_{n=1}^N \sum_{t=1}^T \ln P_{\theta}\{Y_{n,t} = y_{n,t} | x_n, Y_{n,1} = y_{n,1} \dots Y_{n,t-1} = y_{n,t-1}\} \\
&= \arg \min_{\theta} -\sum_{n=1}^N \sum_{t=1}^T \ln \prod_{v=1}^V (\hat{p}_{n,t,v})^{y_{n,t,v}} \\
&= \arg \min_{\theta} -\sum_{n=1}^N \sum_{t=1}^T \sum_{v=1}^V y_{n,t,v} \ln \hat{p}_{n,t,v},
\end{aligned}$$

где је

$$y_{n,t,v} := \begin{cases} 1, & \text{ако се у опису } y_n \text{ на месту } t \text{ налази реч } v \\ 0, & \text{иначе,} \end{cases}$$

а $\hat{p}_{n,t,v}$ процена модела (вероватноћа) да се у опису y_n на месту t налази реч која је у вокабулару индексирана са v . $\hat{p}_{n,t,v}$ зависи од параметара модела θ .

$H(y_{n,t}, \hat{p}_{n,t}) = -\sum_{v=1}^V y_{n,t,v} \ln \hat{p}_{n,t,v}$ је крос ентропија (cross-entropy) за реч на месту t у опису n -тог узорка у скупу података $y_{n,t}$ и предикцију модела $\hat{p}_{n,t}$, по дефиницији унакрсне крос ентропије.

$H(y_n, \hat{p}_n) = -\sum_{t=1}^T \sum_{v=1}^V y_{n,t,v} \ln \hat{p}_{n,t,v}$ је крос ентропија за лабеле скупа података y_n и предикцију модела $\hat{p}_n$ за n -ти узорак, по дефиницији унакрсне крос ентропије.

Закључујемо да је оптимизациони проблем еквивалентан минимизирању суме крос ентропија $H(y_n, \hat{p}_n)$ по узорцима.

$$\begin{aligned}
\theta^* &= \arg \max_{\theta} L(\theta; D) \\
&= \arg \min_{\theta} -\sum_{n=1}^N \sum_{t=1}^T \sum_{v=1}^V y_{n,t,v} \ln \hat{p}_{n,t,v} \\
&= \arg \min_{\theta} \sum_{n=1}^N H(y_n, \hat{p}_n)
\end{aligned}$$

Функција цене $J(\theta)$ за цео скуп примера је збир крос ентропија узорка:

$$J(\theta) = \sum_{n=1}^N H(y_n, \hat{p}_n)$$

Параметри θ^* који максимизирају веродостојност $L(\theta; D)$ су параметри који минимизирају функцију цене $J(\theta) = \sum_{n=1}^N H(y_n, \hat{p}_n)$.

3.3.4 Градијентни спуст

Оптималне вредност параметара θ се налази нумерички, коришћењем градијентног спуста ("*gradient descent*"):

$$\theta_{new} = \theta_{old} - \alpha \nabla J(\theta_{old}),$$

где је $\nabla J(\theta)$ градијент функције цене $J(\theta)$ по параметрима θ , а α је дужина корака која се одређује коришћењем Адам алгоритма (Kingma 2014.).

Тренирање се врши у *batch*-евима у више епоха ради боље генерализације.

3.4 Генерисање описа

Генерисање описа за дату улазну слику користи натренирани енкодер и *greedy* методу узорковања натренираног декодера.

Нека је x препроцесуирана слика за тестирање. Да би се генерисао опис, у енкодер се проследи слика x као улаз и добија се вектор карактеристика z као излаз. z се прослеђује *LSTM*-у декодера као улаз, а он избацује вредности стања које користимо као иницијалне за генерисање речи. Ембедовани специјални токен за почетак, означен са y_1 , се прослеђује *LSTM*-у декодера заједно са иницијалним вредностима стања као улаз и он производи предвиђање за прву реч описа $\hat{r}_1 \in R^V$ као излаз. Индекс максималне вредности вектора $\hat{r}_1$ је индекс прве предвиђене речи w_1 . У следећем кораку се w_1 ембедује и понавља се описна процедура, све док предвиђена реч није специјални токен који означава крај, или до кад се не достигне максимални број речи у реченици.

3.5 Евалуација

За евалуацију се користи скуп за тестирање. Аутоматски генерисани опис се упоређује са скупом људски аотираних референтних описа коришћењем *BLEU* метрике детаљно објашњене у глави 2.5.

Глава 4

Имплементација

Имплементација модела описаног у овом раду је доступна на <https://github.com/andjadenic/ImageCaptioning>.

За имплементацију описаног пројекта генерисања описа слика, коришћени су програмски језик Python и следећи пакети:

- **PyTorch** пружа неопходне алате за рад са неуронским мрежама, њихово креирање, тренирање и евалуацију, уз снажно GPU убрзање (Paszke и др. 2019.). Коришћен је за имплементацију модела енкодер-декодер и током свих фаза: тренирања, закључивања (енгл. *inference*) и евалуације.
- **matplotlib** за креирање визуелизација
- **PIL**, **skit-image** и **torchvision** за учитавање, манипулацију и обраду слика
- **statistics** за рачунање статистика у *EDA-y* (*Exploratory Data Analysis*)
- **pycocoevalcap** за евалуацију модела

Свака фаза у пројекту је описана посебно: **Подаци** у секцији 4.1, **Препроцесуирање** у секцији 4.2, **Имплементација модела** у секцији 4.3, **Тренирање модела** у секцији 4.4 и **Евалуација модела** у секцији 4.5.

4.1 Подаци

COCO 2017 скуп података има преко 118.000 примера за тренирање и 5.000 примера за валидацију.

COCO скуп података се може ручно преузети са веб странице <https://cocodataset.org/#download>. За преузимање података, потребно је преузети *zip* датотеке „2017 Train images [118K/18GB]“ и „2017 Val images [5K/1GB]“, као и датотеку са аотацијама „2017 Train/Val annotations [241MB]“ са веб странице <https://cocodataset.org/#download>.

4.2 Препроцесуирање

Preprocessing pipeline слике, детаљно описан у поглављу 3.1, је имплементиран коришћењем пакета PIL, scikit-image и torchvision.

Вокабулар је инстанца класе Vocabulary, направљена коришћењем описа из скупа за тренирање. Класа Vocabulary садржи речнике word2idx и idx2word као атрибуте, којима су дефинисано пресликавање из скупа речи у скуп индекса и његово инверзно пресликавање.

Preprocessing pipeline, детаљно описан у поглављу 3.1, је имплементиран коришћењем инстанце вокабулара и основних структури и функција Python-а.

4.3 Дефинисање модела

Енкодер и декодер су две класе које наслеђују torch.nn.Module и узимају вредности хиперпараметара (*feature size*, *hidden size* и *number of layers*) при иницијализацији.

Трениран је модел у којем је 256 вредност димензије ембедованог простора слика и речи (*feature size*), као и вредност димензије стања LSTM модела (*hidden size*). Модел има један LSTM слој.

При иницијализацији сваки модел је пребачен на GPU.

Испод је приказан број параметара, број параметара који се уче, као и меморија коју они заузимају у **енкодеру**.

- Укупан број параметара: 58,668,864
- Меморија коју заузимају сви параметри: 223.80 MB
- Укупан број параметара који се уче: 525,056
- Меморија коју заузимају параметри који се уче: 2.00 MB

Испод је приказан број параметара, број параметара који се уче, као и меморија коју они заузимају у **декодеру**.

- Укупан број параметара: 15,442,837
- Меморија коју заузимају сви параметри: 58.91 MB
- Укупан број параметара који се уче: 15,442,837
- Меморија коју заузимају параметри који се уче: 58.91 MB

4.4 Тренирање модела

Тренирање се ради у 10 епоха градијентног спуста, за дужину корака се користи Adam алгоритам (Kingma 2014.) са кораком учења 10^{-3} . Величина *batch*-а је 64. У свакој епохи се рачуна просечна функција цене на скуповима података за

тренирање и валидацију. Чува се модел који има најмању просечну функцију цене на скупу података за валидацију. Сачувани модел се користи за закључивање и евалуацију.

Модел је трениран 10 сати на рачунару са NIDIA Quadri RTX 4000 графичком картицом са 8GB, процесором са 8 језгра (енгл. *cores*) и 16 нити (енгл. *threads*) и 64GB оперативне (енгл. *RAM*) меморије.

4.5 Евалуација модела

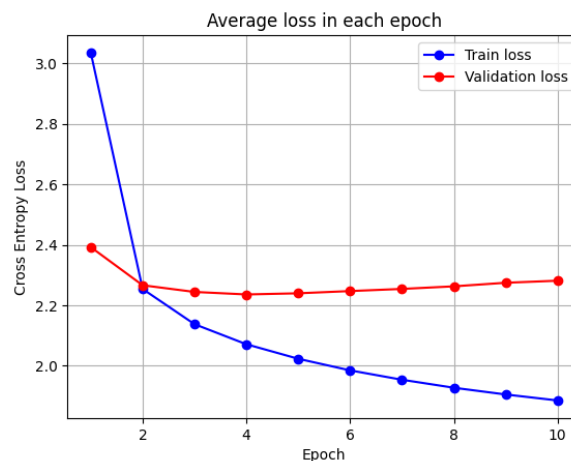
Истренирани модел се евалуира на тест скупу података. Бележе се вредности следећих метрика: *BLEU-1*, *BLEU-2*, *BLEU-3*, *BLEU-4*, *ROUGE-L* и *CIDEr*.

Глава 5

Експерименти

У овој глави су приказани резултати модела.

График 5.1 приказује просечну вредност функције цене на скупу за **тренирање** и **валидацију** у свакој епохи.



Слика 5.1: просечну вредност функције цене на скупу за **тренирање** и **валидацију** у свакој епохи

У табели 5.1 су приказане вредности метрика *BLEU-1*, *BLEU-2*, *BLEU-3*, *BLEU-4*, *ROUGE-L* и *CIDEr* на скупу података за **тестирање**.

Табела 5.1: Вредност евалуационих метрика на скупу података за тестирање у процентима

BLEU-1	BLEU-2	BLEU-3	BLEU-4	ROUGE-L	CIDEr
68.36	50.35	35.70	25.23	49.51	82.62

Издајамо три слике 5.2, 5.3 и 5.4 из скупа за **тестирање** за које је модел генерисао исправан опис.

Издајамо три слике 5.5, 5.6 и 5.7 из скупа за **тестирање** за које је модел није генерисан исправан опис.



Слика 5.2: Генерисан опис: "A person riding a motorcycle on a dirt road."



Слика 5.3: Генерисан опис: "A man is swinging a baseball bat at a game."



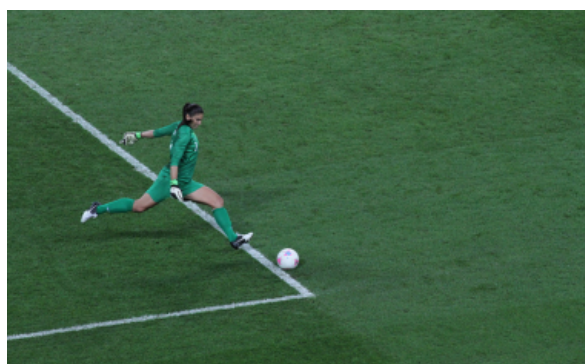
Слика 5.4: Генерисан опис: "A bike parked next to a tree."



Слика 5.5: Генерисан опис: "A motorcycle is parked in front of a building."



Слика 5.6: Генерисан опис: "A table with a bowl of fruit and a bowl of fruit."



Слика 5.7: Генерисан опис: "A woman in a blue shirt is playing tennis."

Глава 6

Закључак и будући рад

Овај мастер рад анализира коришћење енкодер (*ResNet*) - декодер (*LSTM*) модела за аутоматско генерисање описа слике. Резултати указују да коришћење овог модела даје добре резултате.

У даљем раду треба испитати да ли коришћење натренираних (енгл. *pre-trained*) ембединга као што су *word2vec*, *GloVe* или *FastText* у декодеру побољшава перформансе модела. Такође, треба испитати да ли бидирекциони *LSTM* модел у декодеру и визуелног механизма пажње (енгл. *visual attention mechanism*) даје боље резултате.

Биографија кандидата

Анђа Денић је рођена 5.7.1999. године у Нишу. Завршила је специјално одељење за ученике надарене за математику при Гимназији "Светозар Марковић" у Нишу 2018. године. Исте године уписује основне студије математике на Природно-математичком факултету, које завршава 2022. године са просечном оценом 9.16, када уписује мастер студије Вештачке интелигенције и машинског учења при катедри за рачунарске науке на истом факултету.

Од 2024. ради на Електронском факултету у Нишу као сарадник у настави.

Области интересовања су јој машинско учење, статистика, примењена математика и анализа података.

Библиографија

- Bahdanau, Dzmitry, Kyunghyun Cho и Yoshua Bengio (2016.). *Neural Machine Translation by Jointly Learning to Align and Translate*. arXiv: 1409.0473 [cs.CL]. URL: <https://arxiv.org/abs/1409.0473>.
- Bengio, Y., P. Simard и P. Frasconi (1994.). ``Learning long-term dependencies with gradient descent is difficult". У: *IEEE Transactions on Neural Networks* 5.2, стр. 157–166. DOI: 10.1109/72.279181.
- Brunton, Steve (16. јун 2024.). *Residual Networks (ResNet) [Physics Informed Machine Learning]*. YouTube video. Steve Brunton. URL: <http://www.youtube.com/watch?v=w1UsKanMatM>.
- Devlin, Jacob и др. (2019.). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv: 1810.04805 [cs.CL]. URL: <https://arxiv.org/abs/1810.04805>.
- Goodfellow, Ian, Yoshua Bengio и Aaron Courville (2016.). *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press.
- He, Kaiming и др. (2016.). ``Deep residual learning for image recognition". У: *Proceedings of the IEEE conference on computer vision and pattern recognition*, стр. 770–778.
- Hochreiter, Sepp и Jürgen Schmidhuber (1997.). ``Long short-term memory". У: *Neural computation* 9.8, стр. 1735–1780.
- Jinning, Heo (2022.). *ResNet*. URL: https://velog.io/@on_board/ResNet (посећено 19. 10. 2025.).
- Johnson, Justin (2021.). *CS231n Convolutional Neural Networks for Visual Recognition: Recurrent Neural Networks*. <https://cs231n.github.io/rnn/>. Accessed: 2025-05-16.
- Kingma, Diederik P (2014.). ``Adam: A method for stochastic optimization". У: *arXiv preprint arXiv:1412.6980*.
- Krizhevsky, Alex, Ilya Sutskever и Geoffrey E Hinton (2012.). ``Imagenet classification with deep convolutional neural networks". У: *Advances in neural information processing systems* 25.
- Kromydas, Bill (2023.). *Convolutional Neural Network: A Complete Guide*. URL: <https://learnopencv.com/understanding-convolutional-neural-networks-cnn/> (посећено 19. 10. 2025.).
- Kunapinun, Alisa и др. (2023.). ``Explainable automated TI-RADS evaluation of thyroid nodules". У: *Sensors* 23.16, стр. 7289.
- LeCun, Yann, Bernhard Boser и др. (1989.). ``Backpropagation applied to handwritten zip code recognition". У: *Neural computation* 1.4, стр. 541–551.

- LeCun, Yann, Léon Bottou и др. (2002.). ``Gradient-based learning applied to document recognition". Y: *Proceedings of the IEEE* 86.11, стр. 2278–2324.
- Lin, Tsung-Yi и др. (2014.). ``Microsoft Coco: Common objects in context". Y: *European conference on computer vision*. Springer, стр. 740–755.
- Olah, Chris (2015.). *Understanding LSTM Networks*. <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>. Accessed: 2025-05-16.
- Paszke, Adam и др. (2019.). ``PyTorch: An Imperative Style, High-Performance Deep Learning Library". Y: *Advances in Neural Information Processing Systems 32*. Curran Associates, Inc., стр. 8024–8035. URL: <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>.
- Radford, Alec и др. (2018.). ``Improving language understanding by generative pre-training". Y.
- Rumelhart, David E, Geoffrey E Hinton и Ronald J Williams (1986.). ``Learning representations by back-propagating errors". Y: *nature* 323.6088, стр. 533–536.
- Russakovsky, Olga и др. (2015.). ``Imagenet large scale visual recognition challenge". Y: *International journal of computer vision* 115, стр. 211–252.
- Sak, Haşim, Andrew Senior и Françoise Beaufays (2014.a). *Long Short-Term Memory Based Recurrent Neural Network Architectures for Large Vocabulary Speech Recognition*. arXiv: 1402.1128 [cs.NE]. URL: <https://arxiv.org/abs/1402.1128>.
- (2014.b). ``Long short-term memory based recurrent neural network architectures for large vocabulary speech recognition". Y: *arXiv preprint arXiv:1402.1128*.
- Simonyan, Karen и Andrew Zisserman (2014.). ``Very deep convolutional networks for large-scale image recognition". Y: *arXiv preprint arXiv:1409.1556*.
- Singh, Vipra (авг. 2023.). *LLM Architectures Explained: Encoder-Decoder Architecture (Part 4)*. https://medium.com/@vipra_singh/llm-architectures-explained-encoder-decoder-architecture-part-4-b96ace71394c. Accessed: 2025-04-20.
- Sutskever, Ilya, Oriol Vinyals и Quoc V. Le (2014.). *Sequence to Sequence Learning with Neural Networks*. arXiv: 1409.3215 [cs.CL]. URL: <https://arxiv.org/abs/1409.3215>.
- Szegedy, Christian и др. (2015.). ``Going deeper with convolutions". Y: *Proceedings of the IEEE conference on computer vision and pattern recognition*, стр. 1–9.
- Tano, Dante (2021.). *Nothing but NumPy -- Understanding & Creating Neural Networks with Computational Graphs from Scratch*. <https://towardsai.net/p/deep-learning/nothing-but-numpy-understanding-creating-neural-networks-with-computational-graphs-from-scratch-6299901091b0>. Accessed: 2025-05-16.
- Tech, Google Cloud (2023.). *Encoder-decoder architecture: Overview*. https://www.youtube.com/watch?v=zbdong_h-x4. YouTube video. URL: https://www.youtube.com/watch?v=zbdong_h-x4.
- Vaswani, Ashish и др. (2023.). *Attention Is All You Need*. arXiv: 1706.03762 [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.

- Vinyals, Oriol и др. (2015.). ``Show and tell: A neural image caption generator''. У: *Proceedings of the IEEE conference on computer vision and pattern recognition*, стр. 3156–3164.
- Xu, Kelvin и др. (јул 2015.). ``Show, Attend and Tell: Neural Image Caption Generation with Visual Attention''. У: *Proceedings of the 32nd International Conference on Machine Learning*. Уредник Francis Bach и David Blei. Св. 37. Proceedings of Machine Learning Research. Lille, France: PMLR, стр. 2048–2057. URL: <https://proceedings.mlr.press/v37/xuc15.html>.



**ПРИРОДНО - МАТЕМАТИЧКИ ФАКУЛТЕТ
НИШ**

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	монографска
Тип записа, ТЗ:	текстуални / графички
Врста рада, ВР:	мастер рад
Аутор, АУ:	Анђа Денић
Ментор, МН:	Александар Трокицић
Наслов рада, НР:	Генерисање описа слике применом LSTM и ResNet модела
Језик публикације, ЈП:	српски
Језик извода, ЈИ:	енглески
Земља публикавања, ЗП:	Р. Србија
Уже географско подручје, УГП:	Р. Србија
Година, ГО:	2025.
Издавач, ИЗ:	ауторски репринт
Место и адреса, МА:	Ниш, Вишеградска 33.
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	67 стр. ; граф. прикази
Научна област, НО:	рачунарске науке
Научна дисциплина, НД:	машинско учење
Предметна одредница/Кључне речи, ПО:	енкодер-декодер, LSTM, ResNet, MC COCO
УДК	004.855
Чува се, ЧУ:	библиотека
Важна напомена, ВН:	

Извод, ИЗ:	Аутоматско генерисање описа слике је фундаментални проблем у вештачкој интелигенцији који повезује рачунарски вид (енгл. Computer Vision, CV) и обраду природног језика (енгл. Natural Language Processing, NLP). У овом раду је представљен генеративни модел заснован на енкодер-декодер архитури, при чему је енкодер ResNet, а декодер LSTM модел. Модел је трениран да за дату слику на улазу максимизира веродостојност анотације на излазу. Експериментално је показано да модел трениран на MC COCO скупу података генерише описе високих перформанси. Модел достиже следеће вредности евалуационих метрика на скупу података за тестирање: 68.36 BLEU-1, 50.35 BLEU-2, 35.70 BLEU-3, 25.23 BLEU-4, 49.51 ROUGE-L, 82.62 CIDEr.
Датум прихватања теме, ДП:	22.10.2025.
Датум одбране, ДО:	
Чланови комисије, КО:	Председник: Члан: Члан, ментор:

Образац Q4.09.13 - Издање 1



**ПРИРОДНО - МАТЕМАТИЧКИ ФАКУЛТЕТ
НИШ**

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	monograph
Type of record, TR :	textual / graphic
Contents code, CC :	master thesis
Author, AU :	Anđa Denić
Mentor, MN :	Aleksandar Trokicić
Title, TI :	Generating image captions using LSTM and ResNet models
Language of text, LT :	Serbian
Language of abstract, LA :	English
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Serbia
Publication year, PY :	2025.
Publisher, PB :	author's reprint
Publication place, PP :	Niš, Višegradska 33.
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	67 p. ; graphic representations
Scientific field, SF :	computer science
Scientific discipline, SD :	machine learning
Subject/Key words, S/KW :	Encoder-decoder, MC COCO, LSTM, ResNet
UC	004.855
Holding data, HD :	library
Note, N :	
Abstract, AB :	Automatic image caption is a fundamental problem in artificial intelligence that connects computer vision (CV) and natural language processing (NLP). In this paper, a generative model based on an encoder-decoder architecture is presented, where the encoder is a ResNet and the decoder is an LSTM model. The model is trained to maximize the reliability of the target caption at the output for a given input image. It is experimentally shown that the model trained on the MC COCO dataset generates high-performance captions. The model achieves the following evaluation metrics on the testing dataset: 68.36 BLEU-1, 50.35 BLEU-2, 35.70 BLEU-3, 25.23 BLEU-4, 49.51 ROUGE-L, 82.62 CIDEr.
Accepted by the Scientific Board on, ASB :	22.10.2025.
Defended on, DE :	

Defended Board, DB :	President:	
	Member:	
	Member, Mentor:	

Образац Q4.09.13 - Издање 1